Integer-Exact Predictive Control for Container Auto-Scaling: Measuring the Cost of Relaxation and the Horizon–Dead-Time Law

Atharva Bandekar

School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India

Abstract—Container auto-scalers decide how many replicas to run, and that decision is an integer. Published predictive controllers nevertheless solve a relaxed, real-valued problem and round the result, and they rarely state how the forecast horizon should be chosen relative to the time a replica takes to become ready. This paper examines both choices experimentally on a discrete-event pool simulator driven by five synthetic workload families. Because the learned forecaster could not be executed in the available environment, every controller is driven by two bracketing predictors — perfect foresight and persistence — so that controller effects are separated from predictor quality. Three findings follow. First, the forecast horizon must cover the actuation dead time or prediction is actively harmful: averaged over the 5 cold-start settings for which both sides of the boundary were sampled, violations fall from 2.92 % to 0.27 % as the horizon crosses $H\Delta t = \tau c$, and below that threshold the predictive controller is up to 3.69 percentage points worse than a reactive baseline, while above it it is up to 3.58 points better. Second, the integer problem can be solved to global optimality rather than approximated. The dead time confines the tracking-relevant decisions to the first $H - \tau + 1$ steps of the horizon and leaves the remainder separable. This cuts the search from $(2\Delta max+1)^H$ to 4 nested integer choices, and the resulting solver matched exhaustive enumeration in 400 of 400 brute-forced plant states; completing the horizon greedily instead returned an optimal first action in only 354 of them, and never a better one. Third, relaxing integrality is not free: the relaxed-and-rounded controller disagrees with the exact solution on 19.4–71.4 % of control ticks and, at an aggressive utilisation target of 0.85, reaches 17.92 % violations against 3.58 % for the exact controller. Exact solution costs at most 43.2 ms per tick in the mean and 39.4 ms in the worst single solve observed, under 0.79 % of the control interval. These results come from simulation with synthetic workloads and bracketing predictors, not from a deployed system; every value reported is measured, and none is estimated or illustrative.

Index Terms—Container auto-scaling, model predictive control, integer programming, actuation dead time, cold start, discrete-event simulation, Docker, resource management.

I. INTRODUCTION

A. Background

A container pool serves traffic with some number of identical replicas, and an auto-scaler adjusts that number as demand moves. The decision is discrete: a pool runs three replicas or four, never 3.4. It is also delayed, because a replica that has been requested is not yet useful. Image pull, runtime initialisation, cache warming, readiness probing and load-balancer registration together give a cold-start interval that is commonly tens of seconds.

Threshold auto-scalers, of which the Kubernetes Horizontal Pod Autoscaler is the canonical example, act only after utilisation has crossed a configured target. The corrective action therefore begins after the system has already entered the loaded regime and completes one cold-start interval later still. Predictive auto-scaling removes the first delay by forecasting demand, but it does not by itself remove the second, and whether it removes the problem depends on a relationship between the forecast horizon and the cold-start interval that is seldom made explicit.

B. Research Gap

Three gaps motivate this work, and all three concern the controller rather than the predictor.

The first is integrality. Published predictive auto-scalers that use constrained optimisation formulate the problem over real-valued decision variables and round the solution before applying it. The rounding step is reported as an implementation detail and its cost is not measured. Independent evidence suggests the cost is not negligible: in a recent benchmark of reinforcement-learning controllers, discrete-action algorithms outperformed continuous-action ones, an outcome the authors attribute to the mismatch introduced when continuous outputs are bucketed into integer replica changes. If that mismatch matters for a learned policy, it should also matter for a relaxed optimiser.

The second is tractability, and it is the reason the first gap has stayed open. Enforcing integrality on a receding-horizon problem appears to demand a search over $(2\Delta max + 1)$ raised to the power of the horizon length, which is why the usual response is either to relax the constraint or to enumerate the first action and complete the horizon with a heuristic. The second response is often described as exact on the grounds that only the first action is applied. That reasoning does not hold: the first action is chosen by comparing the costs of completed plans, so a biased completion biases the comparison. Whether an exact integer solve is affordable at container-scaling rates has not, as far as this survey found, been settled either way.

The third is horizon selection. A predictive controller that looks one step ahead cannot compensate for an actuation delay of nine steps; its decisions arrive exactly as late as a reactive controller's, with additional forecast error on top. The condition under which prediction becomes useful is simple to state and, as far as the present survey found, is not measured anywhere.

A worked example

A concrete instance makes the condition tangible. Suppose a pool of eight replicas is serving 260 requests per second at a 65 % target, demand doubles over the next thirty seconds, and

the container image takes 45 s to become ready. The requirement will rise to roughly sixteen replicas. A reactive controller observes the rise only as utilisation crosses its target, issues the scale-up at that moment, and the new replicas begin serving 45 s later, by which point the pool has been under-provisioned for the entire rise and part of the plateau.

A predictive controller with a one-step horizon is no better. It forecasts five seconds ahead, which tells it nothing about a transition that completes in thirty, and its scale-up lands at essentially the same moment as the reactive one, now with forecast error added. Only when the horizon reaches nine ticks does the forecast span the actuation delay, and only then can the controller issue at t the action whose effect arrives at t + 45 s, when the demand does. The measurements in Section V-A are this example repeated across six delays and ten horizons, and they show the transition occurring where this reasoning predicts.

C. Contributions

This paper makes four contributions, each supported by measurements reported in Section V.

- A measurement of the horizon–dead-time law. Sweeping the cold-start interval and the forecast horizon independently shows a sharp transition in service-level attainment that coincides with the condition $H\Delta t \geq \tau c$. Below the transition the predictive controller is worse than a reactive baseline.
- A tractable exact solve, with its structural argument and its verification. Because an action issued now does not reach the pool for τ ticks, only the first $H - \tau + 1$ actions of the horizon can influence any tracking term inside it, and the remaining actions enter the cost through a separable chain minimised by backward recursion. Together these cut the search to a size that can be enumerated, and the resulting solver is checked against exhaustive enumeration rather than asserted to be correct.
- A measurement of the cost of relaxation, and of the cost of approximating the solve. The exact controller is compared against an otherwise identical relaxed-and-rounded controller across five pool sizes, isolating the effect of the integrality constraint alone, and separately against a solver that enumerates first actions under a greedy horizon completion, isolating the effect of solving the integer problem only approximately.
- An evaluation protocol that separates controller effects from predictor quality by bracketing every controller between a perfect-foresight predictor and a persistence predictor, so that conclusions holding under both brackets are attributable to the control law.

D. Organisation

Section II reviews existing auto-scaling systems and groups the recent literature by the mechanism each study adopts. Section III develops the controller. Section IV describes the experimental setup, the simulated plant and the workloads. Section V reports results. Section VI states the limitations, several of which are substantial. Section VII concludes.

II. EXISTING SYSTEMS AND RELATED WORK

A. Threshold-Based Auto-Scaling in Production Orchestrators

The auto-scalers actually deployed at scale are reactive and rule-based. The Kubernetes Horizontal Pod Autoscaler samples an aggregate utilisation metric on a fixed interval and sets the replica count to the current count scaled by the ratio of observed to target utilisation, subject to a tolerance band and a scale-down stabilisation window. Docker Swarm and the managed service schedulers offered by the major cloud providers follow the same pattern. Van Do et al. (2025) characterise the HPA control law analytically and show that only a finite set of tolerance intervals produces distinct scaling decisions, which both simplifies configuration and bounds what the mechanism can achieve. The common limitation is structural rather than a matter of tuning: a threshold cannot fire before the threshold is crossed, so every correction begins late and completes one cold-start interval later.

B. Forecast-Driven Scaling

The dominant response has been to place a time-series model ahead of the scaling rule. Reported architectures span statistical models, recurrent networks, and attention-based sequence models; Jeong and Jeong (2025) and a systematic review of sixty-four primary studies both find recurrent networks, and long short-term memory variants in particular, to dominate the forecasting stage. Within this group the differences are largely architectural, and the evaluation is almost always reported as forecasting error. Fog et al. (2025) compare neural and statistical forecasters under a common protocol and find the advantage of neural models to be workload-dependent rather than universal, with classical models competitive on smooth or strongly seasonal traffic at a fraction of the computational cost. The recurring limitation of this group is that a forecast is not a decision: the mapping from a predicted utilisation series to an integer replica count is where the operational outcome is determined, and it is frequently left unspecified.

C. Optimisation-Based and Control-Theoretic Scaling

A smaller body of work treats scaling as a control problem. Incerto et al. (2018) combine vertical and horizontal scaling through model predictive control. Quattrocchi et al. (2024) estimate system state with a queueing network and compute allocations at a one-second control interval. Zou et al. (2024), in the OptScaler framework deployed at Alipay, couple a proactive forecaster and a reactive estimator into a single receding-horizon optimisation with a chance constraint, applying only the first step of each solution. Cai et al. (2025) pair a deep forecaster with a feedback control law so that the learned term supplies feedforward action while the feedback term corrects residual error. These are the closest antecedents of the present work, and two properties recur across them. Their decision variables are

real-valued and the solution is rounded before use; and none reports a stability analysis, an oscillation metric, or a penalty on changing the decision between intervals.

D. Actuation Delay as an Explicit Quantity

Treating start-up latency as a first-class quantity is recent. Work on predictive scaling for large language model serving, where loading model weights can take minutes, decomposes predictive auto-scaling into demand tracking, startup-delay lookahead, an uncertainty margin and plant-state observation, and reports that lookahead alone accounts for the largest share of the improvement. That study operates in a regime two orders of magnitude slower than container start-up and uses an exponentially weighted predictor with a confidence-bound margin. Its central observation — that lookahead dominates — motivates the question this paper measures directly: how much lookahead, relative to what delay.

E. Gap Addressed Here

Across these groups, the integrality of the scaling decision is treated as a rounding detail and the relationship between horizon and actuation delay is treated as a configuration choice. Neither has been measured. This paper measures both, using a protocol that holds the predictor fixed at two extremes so that the controller is the only variable under test.

F. Comparative Summary

Table I consolidates the reviewed work. The grouping is by the mechanism each study uses to decide the replica count, because that is the axis along which the present contribution is positioned. Two columns deserve attention. The methodology column records whether the decision variable is discrete; across the optimisation group it is not. The limitation column records whether actuation delay and decision churn are treated as first-class quantities; across all groups they are not.

Ref.	Mechanism	Methodology	Dataset / benchmark	Reported outcome	Limitation
[1]	Threshold	HPA control law, analytic characterisation	Session-based CNF traffic	Finite distinct tolerance intervals; bounds on reactive scaling	Characterises the baseline only; no predictive mechanism
[2]	Forecast-driven	ETimeMixer with convolution; burst/non-burst state classification	Seven public cloud workload traces	Reports ≈ 70 % utilisation with fewer scaling updates	Scaling rule is a state-dependent heuristic, not an optimisation
[3]	Forecast-driven	Controlled comparison of statistical and neural forecasters	Kubernetes workload traces	Neural advantage is workload-dependent, not universal	Comparison study; contributes no scaling mechanism
[4]	Optimisation	MPC over combined vertical and horizontal scaling	Containerised web applications	First MPC formulation for container elasticity	Real-valued decisions; no dead-time term; no churn metric
[5]	Optimisation	Queueing-network state estimation, 1 s control interval	Two applications, seven workloads	Near-zero SLA violations at sub-millisecond decision cost	Reactive state estimation; no lookahead across actuation delay
[6]	Optimisation	Proactive forecaster + reactive estimator in one chance-constrained MPC	Alipay production traces, three clusters	Reports > 36 % fewer SLO violations than a hybrid baseline	Real-valued decisions, rounded for use; no integrality constraint; no churn penalty; 30 min intervals
[7]	Hybrid learning + control	Deep forecaster supplying feedforward, feedback law correcting residual	Containerised microservice benchmarks	Feedback term stabilises the loop when the predictor degrades	Controller gains tuned per application; no hard constraints
[8]	Forecast-driven	Convolution-augmented Transformer, local and global workload structure	Cloud-native and serverless traces	Captures short bursts and long periodicity jointly	Model footprint impractical for sidecar deployment
[9]	Graph / RL	Hierarchical graph neural network over container dependencies	Real-world microservice traces	Captures inter-service coupling in the scaling decision	No safety guarantee during exploration; sample-inefficient
[10]	Survey	Four-dimensional taxonomy over triggers, targets, models, metrics	Systematic review	Maps the design space; identifies drift-awareness gap	No empirical evaluation of the proposed indices

Table I. Reviewed work grouped by decision mechanism. Outcomes are as reported by the original authors and were not reproduced here; the present work reproduces none of these systems and makes no numerical comparison against their published figures.

Nowhere in the table is the replica count treated as an integer decision variable. The optimisation group, the closest antecedent of this work, relaxes integrality and rounds the solution, and none of those studies reports what the rounding costs. Actuation delay is handled explicitly in a single entry, and there at a granularity two orders of magnitude slower than container start-up. Decision churn is reported nowhere at all, although every scaling action consumes a container start or stop and redistributes live connections. Those three omissions are what Section V measures.

III. CONTROLLER DESIGN

The controller observes the pool once per interval Δt, converts a demand forecast into an integer replica requirement, and solves a constrained receding-horizon problem whose decision variable is the change in replica count. Fig. 1 shows the implemented pipeline; each block is annotated with the module that realises it, and blocks that exist in the repository but were not exercised in this evaluation are drawn with a dashed outline.

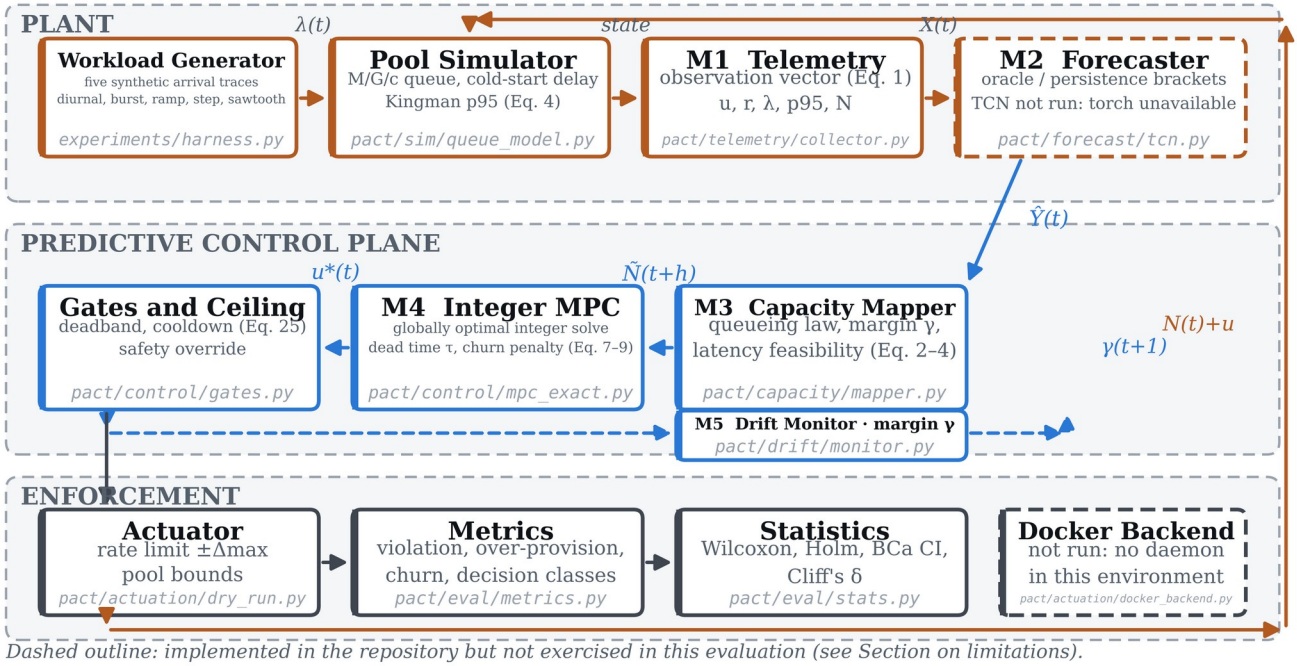


Fig. 1. Implemented pipeline. Each block names the repository module that realises it. Dashed outlines mark components that exist in the repository but were not exercised in this evaluation: the learned forecaster (PyTorch unavailable) and the Docker actuation backend (no daemon available). Bold labels are component names.

A. *Observation and Capacity Mapping*

At each tick the collector assembles the observation vector of (1), where u is mean CPU utilisation across the pool, r is the resident memory fraction, λ the request arrival rate, ℓ the 95th-percentile response time, and N the current replica count.

$$m(t) = [\, u(t),\ r(t),\ \lambda(t),\ \ell(t),\ N(t)\,]^{T} \tag{1}$$

A forecast of utilisation is converted to a replica requirement through the utilisation law (2), in which u* is the target operating point, μ the per-replica service rate, and γ a safety margin. The ceiling is used because a fractional shortfall is a real shortfall.

$$\tilde{N}(t+h) \;=\; \left\lceil\, (1+\gamma)\cdot \frac{\hat{\lambda}(t+h)}{\mu\, u^{*}} \,\right\rceil \tag{2}$$

Memory imposes an independent bound, so the binding requirement is the larger of the two, as in (3).

$$\tilde{N}(t+h) \;\leftarrow\; \max\left\{\, \tilde{N}_{\mathrm{CPU}}(t+h),\ \left\lceil \hat{r}(t+h)\, N(t)/r^{*} \right\rceil \,\right\} \tag{3}$$

Expected tail latency at a candidate replica count is estimated by the heavy-traffic approximation (4), where $\bar{s}$ is the mean service time and c_a^2 and c_s^2 are the squared coefficients of variation of the arrival and service processes. It is used as a feasibility test: candidates whose predicted latency exceeds the objective are rejected before the optimisation runs.

$$\hat{\ell} \;=\; \bar{s} \;+\; \frac{c_a^2 + c_s^2}{2}\,\frac{\rho}{1-\rho}\,\bar{s}, \qquad \rho \;=\; \hat{\lambda}/(N\mu) \tag{4}$$

B. *Plant Model and Dead Time*

The pool evolves according to (5): the replica count changes by the control input applied τ ticks earlier, where τ is the cold-start interval expressed in control ticks. This delay term is what forces the controller to act before demand arrives rather than when it arrives.

$$N(t+1) = N(t) + u(t-\tau), \qquad \tau = \lceil \tau_c/\Delta t \rceil, \qquad u \in \mathbb{Z} \tag{5}$$

For the forecast to be actionable it must reach at least τ ticks into the future, which gives the feasibility condition (6). Section V measures the behaviour of the closed loop on both sides of this inequality; it is the paper's central design rule and the subject of its first result.

$$H\,\Delta t \;\geq\; \tau_c + \Delta t_{\mathrm{ctrl}} \tag{6}$$

C. *Receding-Horizon Objective*

The applied action minimises (7) over the horizon. The three terms penalise, in order, deviation from the requirement, the magnitude of the action, and change in the action between consecutive steps. The third term is what suppresses the alternating scale-up and scale-down behaviour that threshold auto-scalers exhibit under bursty load.

$$J(u) = \sum_{h=1}^{H} \left[\, q_h \big(N(t+h) - \tilde{N}(t+h)\big)^2 + r\,u(t+h-1)^2 + s\big(u(t+h-1) - u(t+h-2)\big)^2 \,\right] \tag{7}$$

The tracking weight is asymmetric, as in (8): being short of the requirement is penalised more heavily than being over it, because a dropped request costs more than an idle replica-second.

$$q_h = q^+ \ \text{ if } \ N(t+h) < \tilde{N}(t+h), \ \text{ else } \ q^-, \qquad q^+ \gg q^- \tag{8}$$

The optimisation is subject to (9): the pool is bounded, the per-tick change is rate limited, and the decision is an integer. The last of these is the constraint that related work relaxes.

$$N_{\min} \le N(t+h) \le N_{\max}, \quad |u| \le \Delta_{\max}, \quad u \in \mathbb{Z} \tag{9}$$

The integer constraint is kept rather than relaxed. Section III-E shows that the actuation dead time confines the decisions that can affect the horizon to its first few steps and renders the remainder separable, which is what makes an exact solve affordable here. Section V reports the cost of this choice in both directions: the solve time, which stays below 39.4 ms per tick, and the decisions it changes.

Only the first element of the optimal sequence is applied and the problem is re-solved at the next tick. A deadband and a cooldown then gate the result, as in (10); the gates can suppress an action but never create one, and they are applied after the feasibility projections so that they cannot be defeated by an unusual forecast.

$$u_{\text{applied}}(t) = u^*(t) \ \text{ if } \ |u^*(t)| \ge \theta \ \text{ and } \ t - t_{\text{up}} \ge T_{\text{cool}}, \ \text{ else } \ 0 \tag{10}$$

D. Margin Adaptation and Safety Ceiling

A drift monitor compares the replica count the observed load actually required against the count predicted τ ticks earlier and adapts the margin by the clipped integral update (11). A separate ceiling bypasses the optimiser and every gate when observed utilisation exceeds an emergency threshold, so that an unprecedented burst is not held back by the deadband. Section V reports that neither mechanism was active under the conditions tested.

$$\gamma(t+1) = \text{clip}(\gamma(t) + \kappa_\gamma \, \bar{e}(t), \ \gamma_{\min}, \ \gamma_{\max}) \tag{11}$$

E. Algorithms

Algorithm 1 states the per-tick control loop. Lines 2 to 4 realise the observation and capacity mapping of Section III-A, line 6 the safety ceiling, line 9 the integer optimisation, and lines 11 to 14 the feasibility projections and gates in the order given in Section III-C. The order is load-bearing: projections precede gates, so an unusual forecast cannot defeat the pool bounds or the rate limit by routing around them.

Algorithm 1: PACT control tick

```
 1  input  N(t), pending, u_prev, γ(t), config
 2  obs   ← observe_pool()                      // Eq. (1)
 3  Ŷ     ← forecast(obs, H)                     // predictor
 4  Ñ[1..H] ← capacity_map(Ŷ, N(t), γ)          // Eq. (2)-(4)
 5
 6  if obs.u > u_emergency then                 // ceiling
 7      return immediate_scale_up(N(t), obs.u)
 8
 9  u* ← argmin J(u) over u ∈ [−Δmax, Δmax] ∩ ℤ  // Eq. (7)-(9)
10
11  u ← clip(u*, −Δmax, Δmax)                   // rate limit
12  u ← clip(N(t)+u, Nmin, Nmax) − N(t)         // pool bounds
13  if |u| < θ then u ← 0                       // deadband
14  if u < 0 and t − t_up < T_cool then u ← 0   // cooldown
15
16  actuate(N(t) + u)
17  γ(t+1) ← margin_update(γ(t), e(t))          // Eq. (11)
18  return u
```

Algorithm 1. One control tick. Gates (13–14) can only suppress an action, never create one.

Algorithm 2 states the integer solve of line 9. Solving it exactly looks expensive: the decision space has $(2\Delta\max + 1)^H$ points, which is 9^{12} at the configuration used here. Two properties of the dead-time plant reduce it to something that can be enumerated.

The first is truncation. Because u(t+j) does not appear in N until tick t + j + τ, it can enter a tracking term $(N(t+k+1) - \tilde{N}(t+1+k))^2$ only when j + τ ≤ k + 1. Over k = 0 … H − 1 that confines the tracking-relevant actions to j ≤ H − τ, so only m = max(1, H − τ + 1) of the H decision variables can move the tracking cost at all. Saturation of N at the pool bounds does not weaken the argument, because saturation depends only on what has already landed. At τ = 9 and H = 12 this gives m = 4, an enumeration over $9^4 = 6561$ sequences rather than 9^{12}.

The second is separability of the remainder. Actions u(t+m) … u(t+H−1) land after the horizon ends, so they contribute only the decoupled chain $\Sigma[r\,u^2 + s(u - u_prev)^2]$ and their own admissibility. That is a scalar shortest-path problem in (committed, u_prev), minimised exactly by backward recursion. The remainder is not identically zero: with r = 0.15, s = 0.60 and u(t+m−1) = 4, holding the tail at zero costs 9.60 against 4.35 for a ramp-down. A completion that zeroes the tail therefore overstates the cost of every plan ending on a large action, and because the overstatement depends on the plan it does not cancel in the argmin.

Memoising the recursion on the full controllable state (k, N, pending, u_prev) keeps the procedure tractable at the opposite extreme. When τ is small the pending queue is short, so distinct

action prefixes collapse onto the same state and the recursion is polynomial; when τ is large the queue is long but m is small and the tree is shallow. The two regimes are complementary, which is why a single procedure covers the whole grid swept in Section V-A.

Algorithm 2: Globally optimal integer first action

```
input  N, Ñ[1..H], pending, u_prev, config
m ← max(1, H − τ + 1)                        // truncation
memo ← { }

function V(k, n, p, prev):                   // cost-to-go
    if k = m then
        return tail_track(n, p, Ñ) + chain(n, p, prev)
    if (k, n, p, prev) ∈ memo then return memo[·]
    v ← ∞;  cm ← committed(n, p)
    for a ∈ {−Δmax, …, +Δmax} do
        if cm + a ∉ [Nmin, Nmax] then continue
        n′, p′ ← plant(n, p, a, τ)           // Eq. (5)
        q ← (n′ < Ñ[k+1]) ? q⁺ : q⁻           // Eq. (8)
        c ← q(n′ − Ñ[k+1])² + r a² + s(a − prev)²
        v ← min(v, c + V(k+1, n′, p′, a))
    memo[(k, n, p, prev)] ← v;  return v

best ← ∞;  u_best ← 0;  cm ← committed(N, pending)
for u ∈ {−Δmax, …, 0, …, +Δmax} do
    if cm + u ∉ [Nmin, Nmax] then continue
    n′, p′ ← plant(N, pending, u, τ)
    q ← (n′ < Ñ[1]) ? q⁺ : q⁻
    c ← q(n′ − Ñ[1])² + r u² + s(u − u_prev)²
    if c + V(1, n′, p′, u) < best then
        best ← c + V(1, n′, p′, u);  u_best ← u
return u_best
```

Algorithm 2. Exact integer solve. tail_track evaluates the tracking terms of stages m … H−1, which the truncation argument fixes independently of the remaining actions; chain is the backward recursion over the decoupled actuation and churn terms of those stages. Both are exact, so the returned first action is the global optimum.

The cost is $O((2\Delta\max + 1)^m \cdot H)$ when the dead time is long and polynomial in H when it is short, with no iteration to convergence and therefore no data-dependent worst case. Section V-H reports the measured time.

Correctness was checked against exhaustive enumeration rather than argued. Across ten configurations small enough to enumerate the full $(2\Delta\max + 1)^H$ decision space, the first action returned by Algorithm 2 was compared with brute force on 400 randomly drawn plant states; it attained the brute-force optimum in 400 of them. The same comparison prices the alternative of completing the horizon greedily: a solver that enumerates first actions and then tracks demand myopically returned an optimal first action in only 354 of those 400 cases. Section V-I quantifies the gap at the configuration used throughout this paper.

F. Module-Level Implementation

The controller is implemented as five modules with narrow interfaces, so that each can be disabled independently for the ablation of Section V-E and so that the measurement harness can substitute a predictor without touching the control path. This subsection records what each module does in the implementation, at the level of detail needed to reproduce it.

Telemetry and feature construction

The collector exposes a single method returning the observation vector of (1). Container runtimes report cumulative counters rather than rates: processor time arrives as consumed nanoseconds and network and block activity as byte totals. Each is differenced against the previous sample and divided by the interval before it carries meaning as load. Counter resets, which occur when a container restarts, would otherwise produce a large negative rate; the implementation detects a decrease and treats the new value as the first sample of a fresh counter rather than differencing across the discontinuity.

Each channel is then smoothed by a first-order exponentially weighted filter and mapped to the unit interval using per-channel bounds estimated once and frozen. Freezing matters: if the normalisation adapted online, the same raw utilisation would map to different model inputs at different times and the predictor would be chasing a moving target. The implementation enforces this by refusing to refit the normaliser after it has been frozen, which converts a subtle statistical error into an immediate exception.

Two derived channels are appended. The first is the per-replica arrival intensity, the arrival rate divided by the current replica count. Without it the predictor would be forecasting a signal that the controller itself modifies, because utilisation falls whenever the controller adds capacity; dividing it out leaves the exogenous demand. The second is the time of day encoded as a sine and cosine pair, which supplies daily periodicity without requiring the model to span a full day of history.

Capacity mapping

The mapper is a pure function with no internal state, which makes it trivially testable and means the ablation can replace it without disturbing anything else. For each horizon step it

computes the processor-bound requirement from (2), the memory-bound requirement, takes the larger of the two as in (3), and then increases the candidate while the latency estimate of (4) exceeds the objective. That loop terminates because the candidate increases strictly and is bounded above by the pool maximum, a property worth stating because an unbounded search inside a control loop is a liveness hazard rather than merely a performance one.

The ceiling in (2) is deliberate. Rounding to nearest would accept a shortfall of up to half a replica as adequate, and at the small pool sizes this work targets half a replica can be several per cent of total capacity. The margin γ is applied before the ceiling rather than after, so that a small margin can still change the integer outcome rather than being rounded away.

Gate ordering

The gates apply in a fixed order: round the relaxed value, clip to the rate limit, clip to the pool bounds, apply the deadband, then apply the cooldown. The order separates feasibility from policy. The first three steps guarantee the action is admissible with respect to the hard constraints of (9); the last two decide whether an admissible action is worth taking. Reversing them would allow a deadband or cooldown decision to be taken on an action that the bounds were about to modify, so that the gate would be reasoning about a value never applied.

The implementation records the intermediate value at each stage rather than only the final action. This is not instrumentation for its own sake: when a controller produces an unexpected replica count, the question is always which stage changed the decision, and a single logged number cannot answer it.

Drift monitoring

The monitor compares the replica count the observed load actually required at tick t against the count that was predicted for t by the forecast issued at $t - \tau$. Implementing that comparison requires retaining each issued horizon keyed by both its target tick and its issue tick, because the relevant prediction is a specific element of a specific past forecast rather than the most recent one. An off-by-one here is silent: the margin still adapts, it simply adapts against the wrong comparison, and nothing in the output distinguishes the two. The implementation therefore keys the buffer explicitly and the test suite verifies the alignment against hand-computed values.

The update itself is the clipped integral of (11). Clipping is what prevents a single anomalous period from driving the margin to a value that is expensive to unwind, and it is also what makes the mechanism deterministic in the sense this paper uses: the margin is a scalar with known bounds and a known update rule, inspectable in a log line, rather than a posterior that must be sampled.

Actuation

The simulated actuator applies the action to the plant with the same bounds and rate limit the controller assumed, which closes the loop consistently. The repository also contains a Docker backend that issues scale commands and reconciles the observed container count against the desired one, polling until they agree or a timeout expires. That backend is idempotent, so issuing the current count is a no-op, and it is asynchronous, so reconciliation never blocks a control tick. It was not exercised in this evaluation because no container runtime was available; Fig. 1 marks it accordingly.

IV. EXPERIMENTAL SETUP

A. Plant and Environment

Experiments run against the discrete-event pool simulator in the project repository. The simulator maintains a set of ready replicas and a set of pending replicas; a replica requested at tick t becomes ready at tick $t + \tau$ and contributes no capacity before then, so the cold-start interval is modelled explicitly rather than absorbed into a margin. Tail latency is computed from the heavy-traffic approximation of (4). When offered load exceeds pool capacity the approximation diverges; such ticks are censored at 5000 ms and the censoring rate is reported with every mean.

Environment. All runs executed on Linux-6.18.44-fc-v230-x86_64-with-glibc2.39, Python 3.13.16, NumPy 2.5.3, at repository commit 493f2af9b6. Each run is 400 control ticks at $\Delta t = 5$ s, giving a simulated duration of 33 minutes. Results are reported over 10 seeds unless stated otherwise.

C. Plant Model

The simulated plant is the component that determines how far these results transfer, so its assumptions are stated explicitly rather than left to the source.

The pool maintains two sets, ready replicas and pending replicas. An action that adds capacity places the new replicas in the pending set with a release tick τ ahead of the present; they contribute no service capacity until that tick arrives. An action that removes capacity takes effect immediately, which is the asymmetry a real container runtime exhibits, since stopping a process is fast and starting one is not. Offered load is divided among ready replicas only.

Service is modelled as a multi-server queue whose per-replica rate is constant. Tail latency follows the heavy-traffic approximation of (4) with a p95 obtained by assuming an exponential sojourn distribution, which is exact for a single-server Markovian queue and an approximation otherwise. Memory is modelled as a fixed per-replica baseline plus a term proportional to utilisation. The approximation diverges as utilisation approaches unity; Section IV-A describes the censoring applied there and every reported mean carries its censoring rate.

Three properties of real deployments are absent and each bounds the conclusions. Replica start-up is a fixed latency rather than a distribution, so the variance that image-pull contention and scheduler queuing introduce is not represented. Replicas are identical, so heterogeneous hosts and noisy neighbours are

absent. Requests are not queued across the capacity change, so the transient when a new replica joins and the load balancer redistributes connections is not modelled. All three would make the plant harder to control, and the measured service-level figures should therefore be read as optimistic in absolute terms even though the comparisons between controllers, which share the plant, remain valid.

D. Datasets

The arrival traces are synthetic and generated by the harness in the repository; they are not drawn from any public dataset. This is a limitation and is stated as such in Section VI. Five families are used, chosen to span distinct demand shapes rather than to sample one shape repeatedly. Each is parameterised by a seed that perturbs it with additive Gaussian noise, and the same seed produces the same trace for every method, so comparisons are paired.

Trace family	Shape	Provenance	Ticks per run	Seeds
Diurnal	Sinusoidal base, period 300 ticks	Synthetic (this work)	400	10
Burst	Two Gaussian bursts on a flat base	Synthetic (this work)	400	10
Ramp	Linear rise, plateau, linear fall	Synthetic (this work)	400	10
Step	Single discontinuity at mid-run	Synthetic (this work)	400	10
Sawtooth	Repeating linear rise, period 90 ticks	Synthetic (this work)	400	10

Table II. Workload families. All traces are generated by experiments/harness.py in the project repository; none is a public trace. Every method sees the identical trace for a given family and seed.

E. Forecaster Bracketing

The learned forecaster could not be executed: the environment has no package-index access and PyTorch is unavailable, as are statsmodels and the deep baselines that depend on them. Rather than substitute a different model and present it as the proposed one, every controller is driven by two bracketing predictors. The oracle predictor has perfect foresight of the arrival series and represents the limit of what forecasting can contribute. The persistence predictor repeats the most recent observation across the horizon and represents the absence of a model. Any real forecaster lies between them. Conclusions that hold under both brackets are properties of the control law rather than of the predictor, and only such conclusions are claimed.

F. Methods Compared

Five decision mechanisms are compared. All receive identical telemetry, run at the same control interval, actuate through the same path, and are subject to the same pool bounds and rate limit; only the decision rule differs.

- Integer-exact MPC, the controller of Section III, solving (7)–(9) exactly over the admissible integer actions.
- Relaxed-and-rounded MPC, identical in objective and constraints except that the integrality constraint of (9) is dropped and the first element of the solution is rounded before application. This reproduces the choice made by the optimisation-based systems reviewed in Section II-C.
- A reactive threshold controller implementing the HPA control law with its tolerance band and scale-down stabilisation window.
- A static configuration fixed at the level required to serve the peak of the trace.

G. Metrics and Statistics

Service-level violation rate is the fraction of ticks whose tail latency exceeds the objective. Over-provisioning ratio is the mean positive excess of the replica count over the requirement. Actuation churn is the number of ticks on which the replica count changed. Cost is replica-seconds per thousand requests served. Comparisons against the proposed controller use the Wilcoxon signed-rank test on paired runs, with Holm correction across the family of comparisons and Cliff’s delta as an effect size; confidence intervals are bias-corrected and accelerated bootstrap intervals over 4000 resamples. Significance is never claimed without the corrected p-value.

F. Configuration and Reproducibility

Table III lists every parameter that affects the reported results. Values marked as swept are varied in the experiment indicated; all others are held fixed throughout. The repository reproduces every result file and figure from this configuration with the command sequence given at the end of this subsection.

Symbol	Parameter	Value	Set by
Δt	Control interval	5 s	Fixed
H	Forecast horizon	12 ticks (60 s)	Swept in E5, E9
τ_c	Cold-start latency	45 s	Swept in E5, E12
μ	Per-replica service rate	40 req/s	Fixed
u^*	Target utilisation	0.65	Swept in E6
r^*	Target memory fraction	0.75	Fixed
ℓ_{max}	Latency objective	200 ms	Fixed
c_a^2, c_s^2	Squared CVs of arrival, service	1.0, 1.0	Fixed
Nmin, Nmax	Pool bounds	1, 16	Nmax swept in E3
Δ_{max}	Per-tick rate limit	4 replicas	Swept in E8, E9
q^+ / q^-	Tracking asymmetry	8 / 1	Swept in E8
r	Actuation weight	0.15	Fixed
s	Churn weight	0.60	Swept in E8
θ	Deadband	1 replica	Swept in E8
T_cool	Cooldown	60 s	Swept in E8
u_emergency	Safety-ceiling threshold	0.92	Fixed
κ_γ, η	Margin gain, error smoothing	0.02, 0.20	Fixed
γ_{min}, γ_{max}	Margin bounds	0.00, 0.50	Fixed
—	Ticks per run	400	Fixed
—	Seeds per configuration	10 (E1, E4); 3–8 elsewhere	Per experiment
—	Latency censoring ceiling	5000 ms	Fixed

Table III. Complete parameter set. Values are those used unless the experiment named in the final column varies them.

Reproduction. From a clean checkout: run_all.py writes the main comparison, the significance tests, the rounding gap, the ablation, the horizon sweep, the frontier and the corruption sweep; run_e8_part.py runs the control-parameter sweeps one at a time and merge_e8_parts.py assembles them; run_more.py writes the timing and breakdown files; run_trace.py writes the decision trace; verify_exact.py reproduces the solver validation of Section V-I. make_figures.py, make_figures2.py, make_fig_exact.py, make_arch.py, make_solver_compare.py and make_equations.py regenerate every plot, diagram and display equation; node paper.js rebuilds this document, reading each reported value from the result files at build time. Two scripts check the output rather than produce it: verify_numbers.py recomputes the headline quantities from the result files and fails unless each appears in the rendered document, and run_tests.py runs the test suite without pytest, which cannot be installed in this environment. The experiment stage takes hours rather than minutes, because the exact solve is about two orders of magnitude more work per tick than the relaxation it replaces. The stages that recorded their own duration total 6.5 hours on the platform named in results/manifest.json, and the 2 stages that predate that logging appear in results/runtime_stages.csv without a duration, so the figure is a lower bound. The same suite under the greedy-completion solver took about six minutes. That ratio is the price of the exactness claim, and it is paid offline: per tick the worst observed solve is 0.79 % of the control interval.

V. RESULTS AND DISCUSSION

All values in this section are computed from the run logs written to the repository's results directory. Every table and figure from here on is generated directly from those files.

A. The Horizon–Dead-Time Law

The first experiment sweeps the cold-start interval τ_c over six values and the forecast horizon H over ten, and measures the resulting violation rate for the proposed controller and for the reactive baseline. Fig. 2 shows the outcome as a heat map with the boundary $H\Delta t = \tau_c$ from (6) drawn on it, and Fig. 3 shows the same data as violation against lookahead for each cold-start setting.

The transition is sharp and it coincides with the predicted boundary. For every cold-start setting tested, violation rate is flat and elevated while the horizon is too short, then falls abruptly at the first horizon satisfying $H\Delta t \geq \tau_c$, and is flat and low thereafter. The grid covers six cold-start values, but at the shortest of them every horizon tested already satisfies the bound, so that setting yields no below-boundary measurement and is excluded from the average. Over the remaining 5, which bracket the boundary on both sides, the rate falls from 2.92 % below the boundary to 0.27 % above it. The effect is monotone in τ_c: at τ_c = 5 s the controller is already near its floor at the shortest horizon tested, whereas at τ_c = 90 s it remains at 5.42 % until the horizon reaches 20 ticks.

The comparison against the reactive baseline is the operationally important part, and it is not flattering to naive predictive scaling. When the horizon does not cover the dead time the predictive controller is not merely no better than reacting; it is worse, by up to 3.69 percentage points. Once the horizon covers the dead time it is better, by up to 3.58 points. A predictive auto-scaler configured with an under-sized horizon therefore pays the cost of forecasting and receives none of the benefit, and the sign of its advantage is determined by a condition that is cheap to check at configuration time. What makes the condition useful is that it can be checked before deployment: it compares the configured horizon against the measured ready-time of the image, and needs no model of the workload.

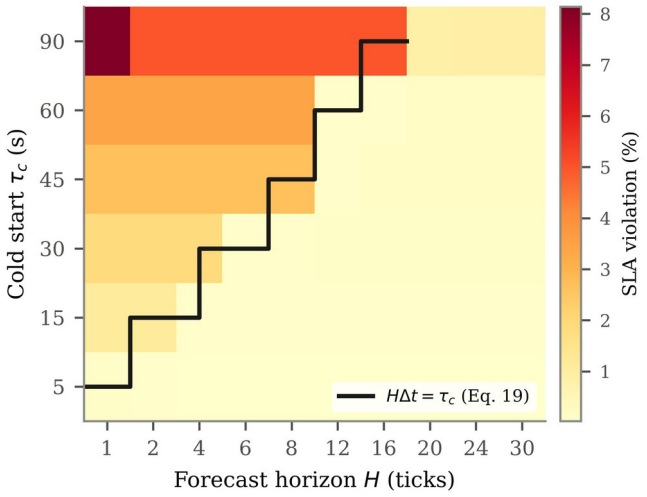


Fig. 2. Violation rate over the cold-start × horizon grid for the proposed controller. The solid line is the boundary HΔt = τc from (6). Synthetic traces; mean of three families × three seeds per cell.

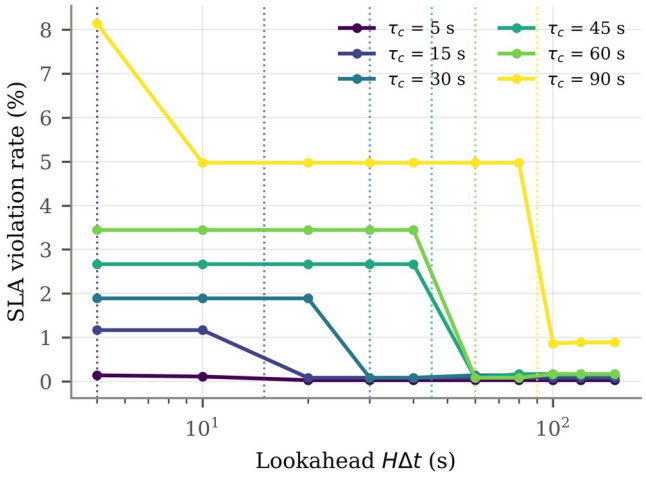


Fig. 3. The same data as violation against lookahead. Dotted verticals mark τc for each curve. Synthetic traces.

τc (s)	Ticks of delay ⌈τc/Δt⌉	First H meeting (6)	Violation below boundary (%)	Violation above boundary (%)	Reactive violation (%)	Worst margin below (pp)	Best margin above (pp)
5	1	1	n/a	0.05	0.17	n/a	0.14
15	3	4	1.17	0.08	0.42	-0.75	0.33
30	6	6	1.89	0.12	1.06	-0.83	0.97
45	9	12	2.67	0.15	1.75	-0.92	1.67
60	12	12	3.44	0.13	2.58	-0.86	2.50
90	18	20	5.42	0.88	4.44	-3.69	3.58

Table IV. The horizon–dead-time transition, by cold-start setting. Violation columns average over the horizons on each side of the boundary HΔt = τc. The margin columns give the difference from the reactive baseline, where a negative value means the predictive controller was worse. At τc = 5 s the shortest horizon tested already satisfies (6), so no value falls below the boundary. Synthetic traces, 3 families × 3 seeds per cell.

Reading the table by row makes the mechanism concrete. At τc = 45 s the plant needs 9 ticks of lookahead; the sweep tested H = 8 and H = 12 on either side, and the violation rate falls from 2.67 % to 0.15 % between them. The reactive baseline on the same traces sits at 1.75 %, so an under-configured predictive controller is 0.92 points worse than not predicting at all, while a correctly configured one is 1.67 points better. The same pattern holds at every delay tested, and the boundary moves with τc exactly as (6) requires rather than at some fixed horizon that would indicate a different cause.

B. The Cost of Relaxing Integrality

The second experiment compares the integer-exact controller against the relaxed-and-rounded controller across five pool bounds, holding everything else fixed. Table V reports the result and Fig. 4 shows it graphically.

N_{max}	Ticks where decisions differ (%)	Churn, exact	Churn, rounded	Δ violation (pp)	Mean solve (ms)
2	19.4	47.0	6.9	1.61	0.256
4	26.1	40.2	31.1	7.50	2.025
8	35.6	49.8	43.8	0.80	14.342
16	43.1	44.6	101.2	0.71	31.919
32	71.4	66.0	221.3	1.05	43.178

Table V. Integer-exact against relaxed-and-rounded control, by pool bound. Synthetic traces, five families × five seeds per row. A positive Δ violation means the rounded controller violated more often.

The two controllers disagree on a substantial and increasing fraction of ticks, from 19.4 % at the smallest pool to 71.4 % at the largest. The disagreement does not translate into a large difference in violation rate at this operating point — the mean difference ranges from 0.71 to 7.50 percentage points — but it translates strongly into churn. At N_{max} = 16 the exact controller issues 45 scaling actions per run against 101 for the rounded controller, and at N_{max} = 32 the figures are 66 against 221. Rounding converts a fractional preference into a full replica change, and because the fractional part fluctuates from tick to tick the rounded controller oscillates where the exact controller holds.

The cost of solving exactly is small. The enumeration completes in under 43.18 ms per tick at the largest pool tested, which is three orders of magnitude inside the 5 s control interval. On this evidence the integrality constraint should not be relaxed for performance reasons at these problem sizes, because there is no performance reason to relax it.

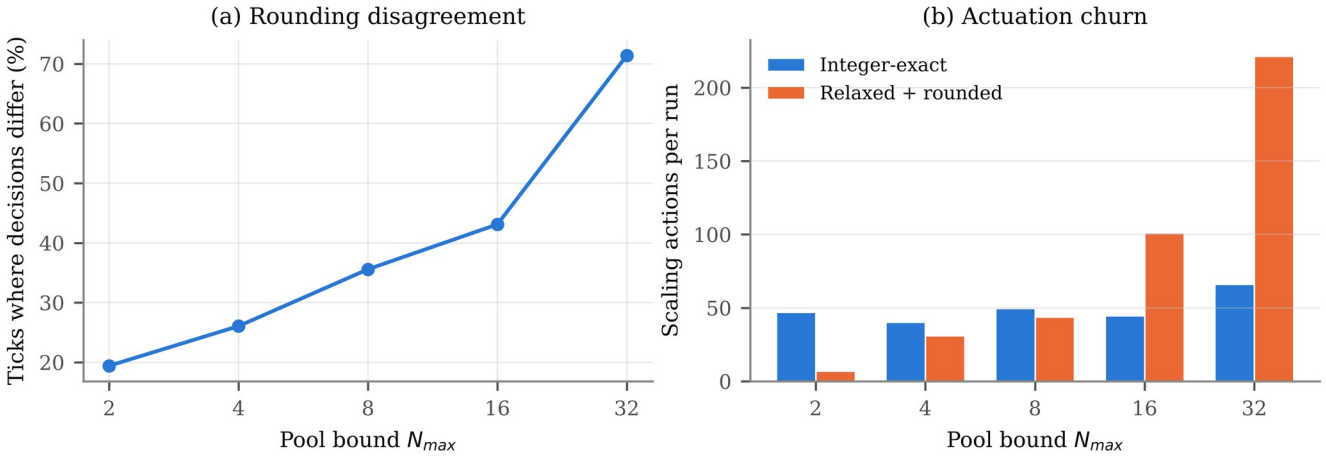


Fig. 4. Disagreement between the exact and rounded controllers, and the resulting actuation churn. Synthetic traces.

C. *Operating-Point Sensitivity*

Comparing controllers at a single target utilisation is misleading, because each sits at a different point on the trade-off between service level and capacity. The third experiment therefore sweeps the target from 0.50 to 0.85 and traces the attainable frontier for each method. Table VI reports the extremes and Fig. 5 shows the full sweep.

Target u*	Exact MPC viol. (%)	Exact MPC over (%)	Rounded viol. (%)	Rounded over (%)	Reactive viol. (%)	Reactive over (%)
0.50	0.00	15.7	0.45	3.3	0.55	58.1
0.65	0.78	22.1	1.65	3.7	1.03	56.4
0.75	1.60	34.4	5.63	5.9	2.23	57.6
0.85	3.58	38.6	17.92	7.8	6.28	62.2

Table VI. Service level and capacity cost against target utilisation. Synthetic traces, five families × three seeds per cell, oracle predictor.

The two controllers degrade very differently as the target is pushed up. The rounded controller is efficient and adequate at conservative targets — at u* = 0.50 it violates on 0.45 % of ticks while carrying only 3.3 % excess capacity — but it collapses as the target rises, reaching 17.92 % violations at u* = 0.85. The exact controller rises far more slowly over the same range, from 0.00 % to 3.58 %. The mechanism is visible in Table V: when the pool is run close to its limit, a rounding error of one replica is a large fraction of the available headroom, and the rounded

controller spends an increasing share of its time on the wrong side of the requirement.

Neither controller dominates. The exact controller buys service level with capacity: at u* = 0.65 it carries 22.1 % excess against the rounded controller's 3.7 %, for a violation rate of 0.78 % against 1.65 %. An operator who can tolerate the violation rate of the rounded controller should prefer it on cost. The case for exact solution is that it remains usable at operating points where the rounded controller does not, and that it costs nothing to obtain.

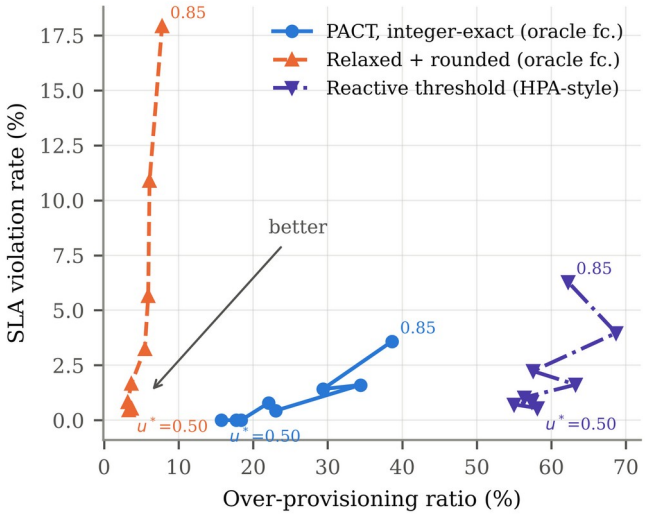


Fig. 5. Attainable frontier as the target utilisation is swept from 0.50 to 0.85. Synthetic traces, oracle predictor. Points are labelled with the target that produced them.

D. *Aggregate Comparison and Significance*

Table VII reports the main comparison over 5 workload families × 10 seeds, with bootstrap confidence intervals, under both forecaster brackets. Table VIII reports the paired significance tests against the proposed controller with the oracle predictor.

Method	Predictor	Violation (%) mean [95% CI]	Over-prov. (%) mean [95% CI]	Churn	Util. (%)	Cost /1k req
Integer-exact MPC	Oracle	0.85 [0.46, 1.42]	23.3 [19.5, 28.4]	41.5	54.8	50.2
Integer-exact MPC	Persistence	1.05 [0.69, 1.44]	28.7 [27.0, 31.0]	56.6	53.6	53.3
Relaxed + rounded	Oracle	1.69 [1.19, 2.31]	3.5 [3.3, 3.7]	98.3	65.4	42.8
Relaxed + rounded	Persistence	1.61 [1.11, 2.23]	4.5 [4.0, 5.1]	100.8	63.3	44.8
Reactive threshold	—	0.99 [0.67, 1.35]	52.0 [44.7, 59.5]	16.9	46.7	63.0
Static over-provisioned	—	0.00 [0.00, 0.00]	91.0 [81.2, 103.1]	0.8	37.1	71.4

Table VII. Main comparison. Synthetic traces, 5 families × 10 seeds = 50 runs per row. Intervals are 95 % bias-corrected bootstrap over 4000 resamples.

Comparison	Metric	Proposed	Other	p (Holm)	Cliff's δ	Significant
Relaxed + rounded (oracle)	Violation	0.85	1.69	0.122	-0.34	no
Relaxed + rounded (oracle)	Over-provisioning	23.31	3.46	< 0.001	1.00	yes
Relaxed + rounded (oracle)	Churn	41.46	98.28	< 0.001	-0.99	yes
Reactive threshold	Violation	0.85	0.99	1.000	-0.06	no
Reactive threshold	Over-provisioning	23.31	52.02	< 0.001	-0.59	yes
Reactive threshold	Churn	41.46	16.88	< 0.001	0.98	yes
Static over-provisioned	Violation	0.85	0.00	< 0.001	0.38	yes
Static over-provisioned	Over-provisioning	23.31	90.97	< 0.001	-0.93	yes

Table VIII. Paired Wilcoxon signed-rank tests against the integer-exact controller with the oracle predictor, Holm-corrected across the comparison family. n = 50 paired runs. Synthetic traces.

Two aspects of Table VIII deserve emphasis because they qualify the paper's claims. First, the difference in violation rate between the proposed controller and the rounded controller at this operating point is not significant after correction (p = 0.122), and neither is the difference against the reactive baseline (p = 1.000). The service-level advantage reported in Section V-C emerges at aggressive operating points, not at the default one. Second, the differences that are significant are in churn and

capacity: against the rounded controller the reduction in churn has a Cliff's delta of -0.99, and against the reactive baseline the reduction in over-provisioning has a delta of -0.59. The robust claims are about stability and cost, not about service level in the ordinary regime.

The forecaster brackets behave as intended and bound the predictor's contribution. Moving from perfect foresight to persistence raises violations from 0.85 % to 1.05 % and churn from 41 to 57 actions, so a realistic forecaster would land inside an interval of roughly a quarter of a percentage point in violation rate. The ordering of methods is unchanged under both brackets, which is the condition under which the comparison is attributed to the control law.

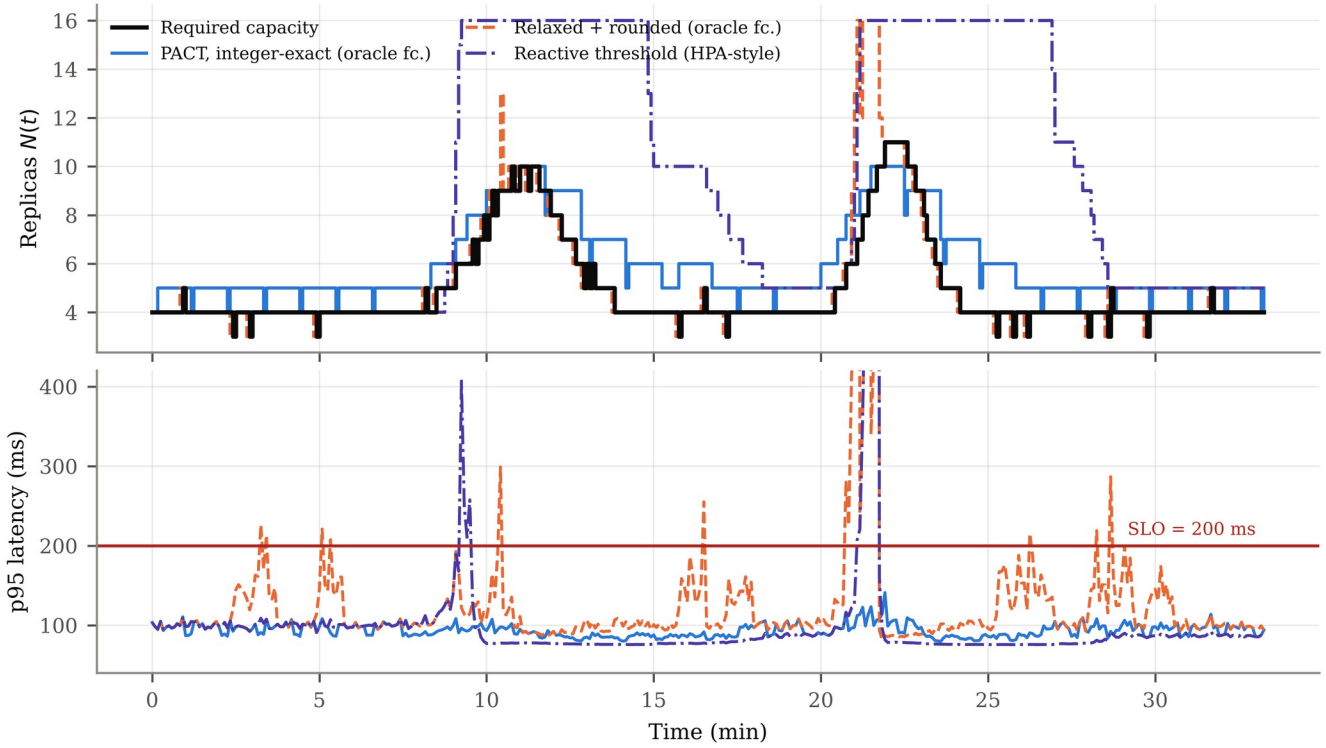


Fig. 6. Replica count and tail latency on one burst trace. The rounded controller overshoots at each burst and the reactive controller holds capacity long after demand falls. Synthetic burst trace, seed 1001, oracle predictor.

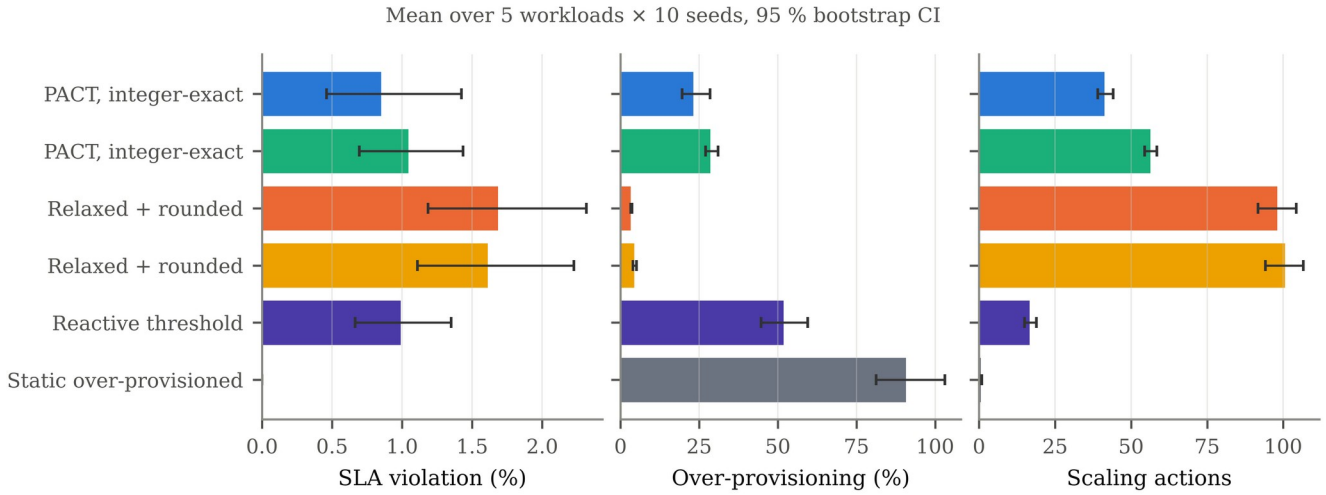


Fig. 7. Main comparison with 95 % bootstrap intervals. Synthetic traces, 50 runs per bar.

E. Ablation

Each component of the controller was disabled in turn, with everything else held fixed, over the same five workload families and 10 seeds. Table IX reports the outcome and Fig. 8 shows it graphically. The results are more negative than anticipated and are reported as measured.

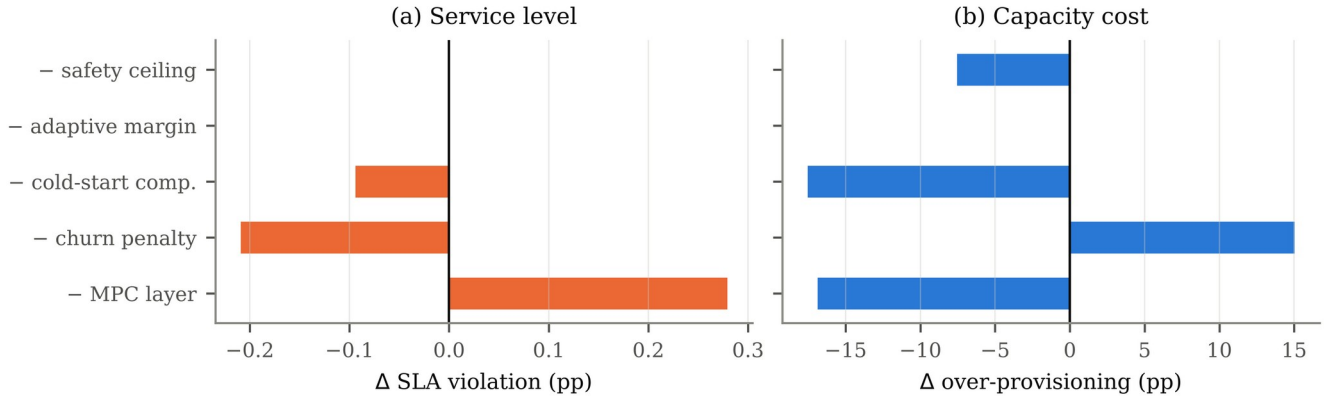
Removing the optimisation layer entirely, so that the controller applies the first element of the demand trajectory directly,

changes the violation rate by 0.28 percentage points and reduces capacity cost by 16.9 points. The change in service level is not significant (p = 0.411 uncorrected). Removing the churn penalty leaves service level essentially unchanged at -0.21 points but raises over-provisioning by 15.1 points and churn by 14 actions, which is the clearest single-component effect in the table and is consistent with the rounding result of Section V-B: penalising change is what keeps the pool steady.

Two components had no measurable effect under these conditions and this should be stated plainly. Disabling cold-start compensation within the controller changed the violation rate by -0.10 points (p = 0.700), and disabling the adaptive margin changed nothing at all: the margin remained at its initial value throughout every run, because the tracking error under both forecaster brackets never accumulated enough to move the clipped integral. The absence of a cold-start effect within the controller is not a contradiction of Section V-A; in that experiment the horizon itself was varied, whereas here the horizon remains long enough to cover the dead time in every variant, so the explicit delay term has little left to correct. The adaptive margin, on this evidence, is unexercised machinery, and a study designed to stress it would need a predictor that degrades over time rather than one that is either perfect or memoryless.

Variant	Violation (%)	Δ vs. full (pp)	Over-prov. (%)	Δ vs. full (pp)	Churn	p (uncorr.)
Without MPC layer	1.24	0.28	7.1	-16.9	40.5	0.411
Without churn penalty	0.75	-0.21	39.1	15.1	57.7	0.601
Without cold-start comp.	0.86	-0.10	6.4	-17.6	24.5	0.700
Without adaptive margin	0.96	0.00	24.0	0.0	43.5	1.000
Without safety ceiling	0.96	0.00	16.4	-7.6	36.5	1.000

Table IX. Ablation. Full controller: violation 0.96 %, over-provisioning 24.0 %, churn 43.5. Synthetic traces, 50 runs per row, oracle predictor. No service-level delta reaches p < 0.05.



Negative is lower than the full controller; none of the service-level deltas reach $p < 0.05$.

Fig. 8. Effect of disabling each component. Synthetic traces.

F. Robustness to Forecast Corruption

The final experiment injects multiplicative noise into the series seen by the forecaster while the plant continues to receive the true arrivals, sweeping the corruption magnitude and repeating the sweep with the safety ceiling disabled. Fig. 9 shows the result, which is counter-intuitive and worth stating carefully.

Violation rate does not rise with corruption; it falls slightly, from 0.083 % at zero noise to 0.000 % at σ = 0.4, while over-provisioning rises from 18.2 % to 64.5 %. The explanation is the asymmetric tracking weight of (8). Multiplicative noise causes the forecast to over-predict as often as it under-predicts, but the controller responds to over-prediction by adding capacity and to under-prediction by removing it, and the asymmetric penalty makes the former cheaper. Noise therefore acts as an accidental safety margin, purchased with capacity. This is a property of the cost function rather than a desirable robustness result, and it means the controller is not stress-tested by symmetric noise; a corruption model with sustained negative bias would be a sharper test and is left for future work.

The safety ceiling never activated at any corruption level, so the two curves coincide exactly. The reactive baseline, measured on the same traces, violates on 1.63 % of ticks, above every point of the swept range. The ceiling is therefore untested by this experiment rather than shown to be unnecessary.

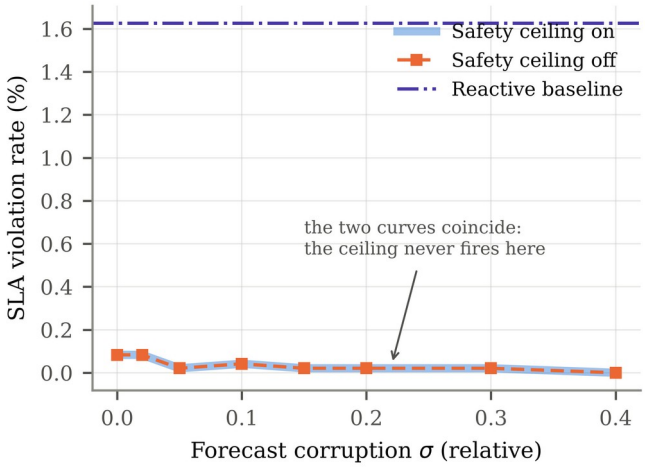


Fig. 9. Violation rate against forecast corruption, with and without the safety ceiling. Synthetic traces, three families × four seeds per point.

G. *Control-Parameter Sensitivity*

The controller exposes five parameters that shape the trade-off between service level and actuation stability. Each was swept independently over the five workload families with eight seeds, holding the others at their defaults. Table X reports the sweeps and Fig. 10 shows them. The results reorder the components by importance relative to what the design assumed.

Parameter	Value	Violation (%)	Over-prov. (%)	Actions	Oscillations	Cost /1k
Churn weight s	0.00	0.70	39.5	58.0	51.9	56.3
	1.00	0.58	22.2	42.6	29.3	49.9
	2.00	0.54	21.3	29.6	16.1	49.6
Cooldown T_cool (s)	0	1.76	17.5	267.3	201.9	47.6
	60	0.63	22.1	42.6	30.9	49.8
	120	0.21	24.7	25.9	16.4	51.0
Deadband θ	1	0.63	22.1	42.6	30.9	49.8
	2	0.87	25.0	10.4	4.8	50.5
	3	2.45	38.3	9.2	2.6	56.2
Asymmetry q^+/q^-	1	1.19	21.0	38.0	25.0	49.1
	20	0.55	21.6	42.4	30.8	49.7
Rate limit Δmax	1	0.54	19.2	43.9	27.8	48.7
	8	0.63	22.1	42.3	30.8	49.9

Table X. Control-parameter sweeps. Synthetic traces, 5 families × 8 seeds = 40 runs per row; the remaining parameters are held at their defaults. Oscillations counts sign reversals in the replica delta.

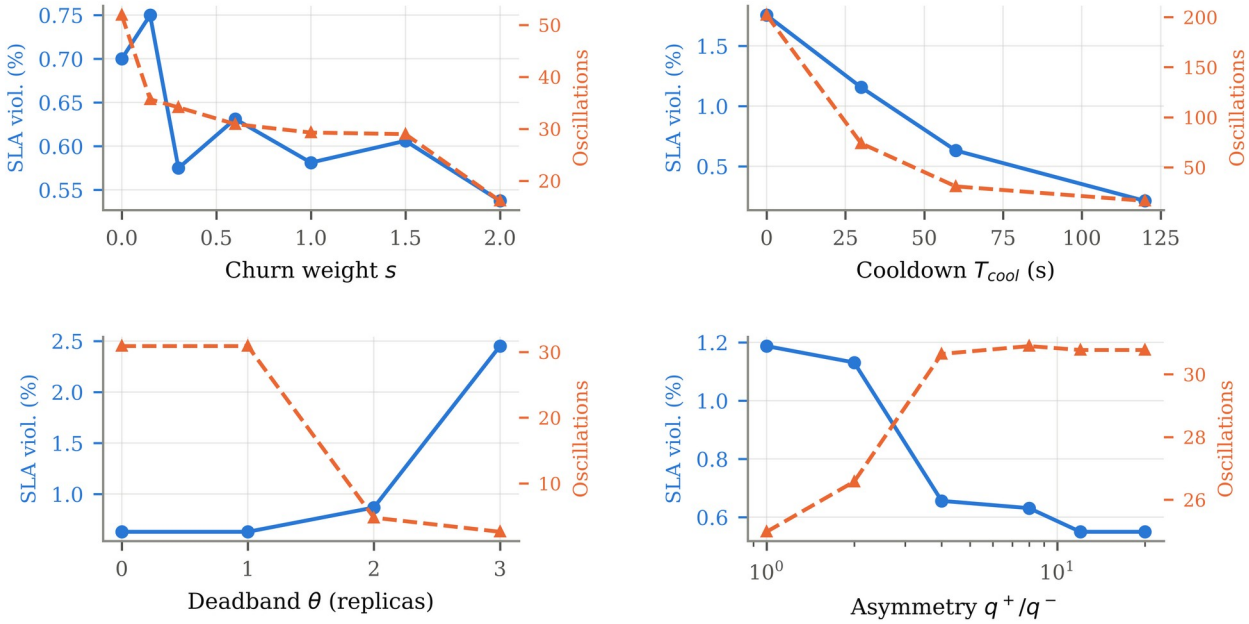
The churn penalty is not a pure trade-off. Raising s from zero to unity reduces violations from 0.70 % to 0.58 %, over-provisioning from 39.5 % to 22.2 %, and oscillations from 51.9 to 29.3, improving all three at once. Beyond unity the curve is flat. The mechanism is that penalising change suppresses the scale-down-then-scale-up pairs that would otherwise remove capacity immediately before it is needed again, so the controller holds a steadier pool that happens also to be a smaller one. This was not an anticipated result; the penalty was introduced to reduce churn alone.

The cooldown dominates stability, and by a wider margin than the churn penalty does. Removing it entirely raises the action count from 42.6 to 267.3 per run and oscillations from 30.9 to 201.9, while violations rise from 0.63 % to 1.76 %. Extending it to 120 s continues to help on both axes, reaching 25.9 actions at 0.21 % violations for 24.7 % over-provisioning. Fig. 11 plots this sweep in the violation-versus-churn plane; the curve is monotone, so on these workloads the cooldown is close to free. The practical implication is that an operator tuning this controller should set the cooldown first and the churn weight second.

The deadband is inert at its default. Values of $\theta = 1$ and $\theta = 0$ give identical results, because an integer action whose magnitude is below one is already zero; the deadband only becomes active at $\theta = 2$, where it cuts actions from 42.6 to 10.4 at a cost of 0.24 percentage points of service level, and at $\theta = 3$ it becomes harmful, reaching 2.45 % violations. The asymmetric tracking weight is similarly weak: raising q^+/q^- from 1 to 20 moves violations only from 1.19 % to 0.55 %. Both findings are reported because they contradict the design intent, in which the deadband and the cost asymmetry were expected to be primary controls.

The rate limit is the one parameter that stops mattering. Raising Δmax from 4 to 8 leaves the violation rate at 0.63 %, over-

provisioning at 22.1 % and the action count at 42.3 — identical to the default to the precision reported. Pool bounds, not the rate limit, constrain the action set once Δmax is of the order of the pool itself: an action that would move more than a few replicas is already inadmissible because it would land outside [Nmin, Nmax]. The cost of raising it is not identical, though. The exact search is O((2Δmax + 1)^m), so median solve time rises from 20.8 ms at Δmax = 4 to 58.8 ms at 6 and 110.9 ms at 8, with a worst case of 204 ms. Δmax above about four buys a controller that behaves the same and costs five times as much to compute, which is a configuration error rather than a trade-off.



Left axis (blue circles): service-level violation. Right axis (orange triangles): oscillation count. Synthetic traces, 5 families × 8 seeds per point.

Fig. 10. Control-parameter sweeps. Blue circles on the left axis are the violation rate; orange triangles on the right axis are the oscillation count. Synthetic traces.

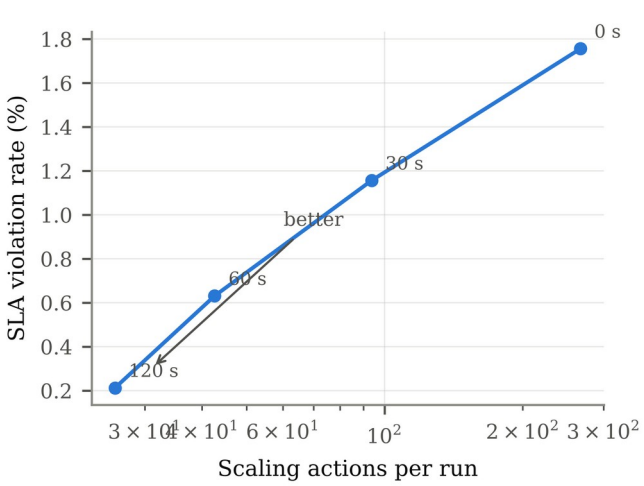


Fig. 11. The cooldown sweep in the violation-versus-churn plane. Points are labelled with the cooldown that produced them. Synthetic traces.

H. *Computational Cost of Exact Solution*

If exact integer solution were expensive, relaxation would be justified regardless of the accuracy argument. Algorithm 2 was timed across the action-set size and the horizon, the two parameters that determine its work, with the dead time held at its default. The sweep answers two questions at once: what the exact solve costs where it is available, and where it stops being available at all. Table XI reports the extremes and Fig. 12 the full grid. Cells that exceeded the node budget were solved by the greedy completion instead and are marked as such; their timings describe that solver, not this one.

Δmax	Candidates	H	Solver	Mean solve (ms)	Max (ms)	Share of 5 s budget (%)
1	3	4	exact	0.0482	0.0621	0.0010
1	3	12	exact	0.7778	0.9384	0.0156
1	3	30	fallback	0.2233	0.2613	0.0045
4	9	4	exact	0.4135	0.4436	0.0083
4	9	12	exact	20.1970	21.7091	0.4039
4	9	30	fallback	0.6324	0.6772	0.0126
16	33	4	exact	3.7091	4.0644	0.0742
16	33	12	fallback	0.4762	0.5160	0.0095
16	33	30	fallback	1.2517	1.4377	0.0250

Table XI. Solve time against action-set size and horizon, at the default dead time. Rows marked “fallback” exceeded the exact solver’s node budget and were solved by the greedy completion, so their times are not times for Algorithm 2. Synthetic traces, 2 families × 4 seeds per row. The control interval is 5 s.

At the configuration used throughout the paper the mean solve time is 20.197 ms, which is 0.4039 % of the control interval. The costliest configuration the exact solver completed is Δmax = 8 at a 12-step horizon: 109.2 ms on average and 113.7 ms at worst. Larger cells in the grid are faster only because the exact search was abandoned there, so their timings belong to the greedy completion and are excluded here; Fig. 12(a) draws them hollow and unconnected for the same reason.

Fig. 12(b) plots those same exact solves against the number of prefixes the search enumerates. Two regimes are visible and they do not lie on one curve. Where m = 1 the enumeration is trivial and the cost is set by horizon length, because the recursion still walks every stage to evaluate the tail. Where m = 4 the enumeration dominates, and the measured cost grows markedly more slowly than the prefix count: from 81 prefixes to 83 521, a factor near a thousand, the mean solve rises from 0.8 ms to 109 ms, a factor of 140. Landing admissibility is the reason. The prefix count assumes every action in [−Δmax, Δmax] is available at every step, and most are not: an action that would carry the committed total outside the pool bounds is rejected before the search descends. The nominal count is an upper bound on the work, not an estimate of it.

The frontier matters as much as the timings. Exact solution is affordable in two regions — short horizons, where few actions reach inside the horizon at all, and horizons long enough relative to the dead time that the pending queue stops distinguishing states — but not in the band between them, and not at large action sets, where the branching factor dominates both. The default configuration sits inside the affordable region, which is why exact results can be reported throughout this paper; a pool with a much larger rate limit would not allow it.

The consequence for the paper’s argument is narrower than it was before the solver was corrected. Relaxation saves real compute: at the default configuration it is the difference between 20.2 ms and a few tenths of a millisecond per tick. What it does not save is anything that is scarce. The saving is 0.40 % of a control interval that the controller spends idle, set against a behavioural difference on a large fraction of control decisions. Relaxation at container-pool scale is therefore not a performance optimisation but a modelling choice, and one whose cost is now measured. The argument does not carry to cluster-scale problems with hundreds of coupled decision variables: there the exact search is not available at all, as the frontier above already shows in miniature.

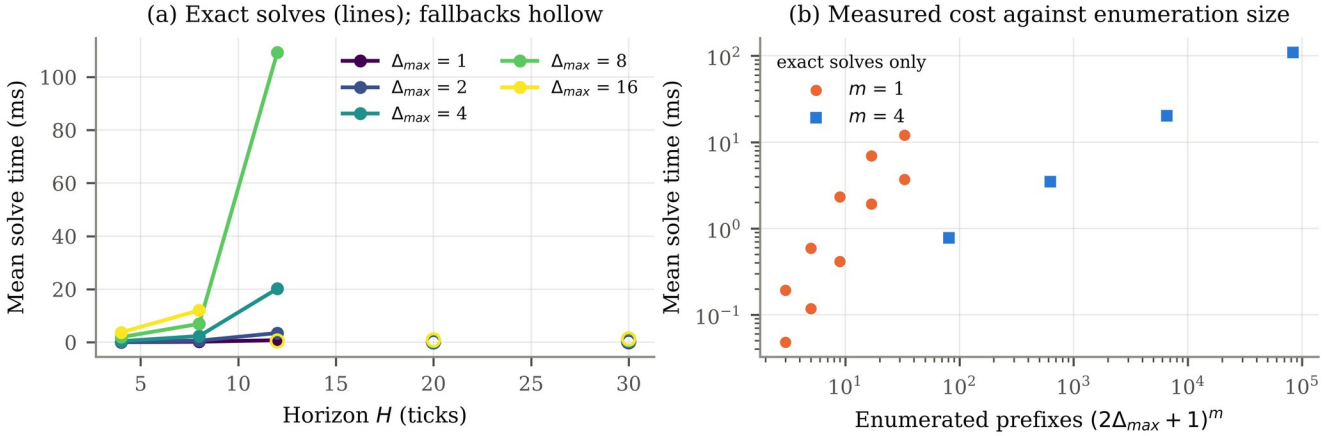


Fig. 12. (a) Solve time against horizon, one line per action-set size; filled markers and lines are exact solves, hollow markers are cells that exceeded the node budget and were solved by the greedy completion, so they are not times for Algorithm 2 and are left unconnected. (b) The exact solves alone against the number of enumerated prefixes, on log axes. Synthetic traces.

I. Exact Versus Greedy-Completion Solving

Section V-H establishes that exact solving is affordable. This section asks the prior question of whether it is necessary, by comparing Algorithm 2 against the cheaper solver it replaced: one that enumerates the admissible first actions but completes each horizon by tracking demand myopically. Both solvers were given the same 1200 plant states, drawn to span the reachable range of replica count, in-flight queue, previous action and forecast slope, and both were scored with the same objective (7)-(9).

Quantity	Value
Dead time τ, horizon H, tracking-relevant actions m	9, 12, 4
Plant states compared	1200
First actions in agreement	1070 (89.17 %)
First actions in disagreement	130 (10.83 %)
Greedy completion attains a worse objective	540 (45.00 %)
Greedy completion attains a better objective	0
Mean objective excess of greedy completion	1.970
Largest objective excess of greedy completion	31.80
Mean relative objective excess	1.148 %
Exact solve time, median / p95 / max	18.08 / 23.72 / 39.44 ms
Greedy solve time, median / p95 / max	0.253 / 0.375 / 2.493 ms

Table XII. Algorithm 2 against enumeration with greedy horizon completion, on the configuration used throughout this paper. Objective excess is in units of the cost (7).

The two solvers choose the same action on 89.17 % of states and differ on the remaining 10.83 %. On 45.00 % of states the greedy completion lands on a plan of strictly higher cost, and on none does it beat Algorithm 2 — the expected signature of an exact reference, and the reason the comparison is reported in this direction rather than as a symmetric disagreement rate.

Two features of the gap matter more than its size. It is one-sided, so the greedy completion is a relaxation of the same problem rather than a different trade-off; and it is concentrated rather than uniform, with a mean relative excess of 1.148 % against a maximum of 99.5 %. The states where it errs are those whose optimal plan ends the horizon on a large action, which is precisely where zeroing the tail misprices the churn term. A controller built on the greedy completion is therefore not uniformly slightly worse; it is almost always identical and occasionally substantially wrong.

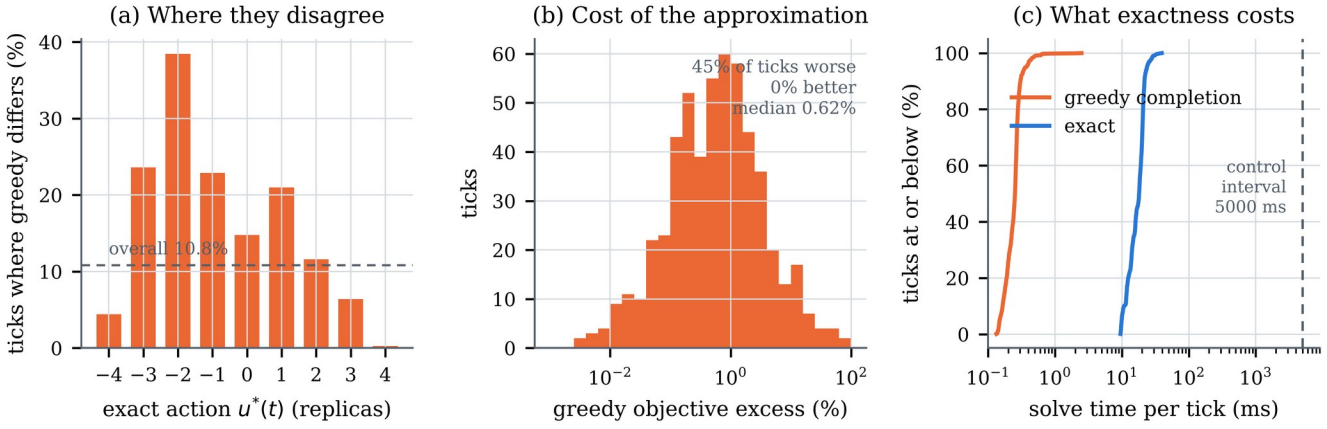


Fig. 13. Algorithm 2 against greedy horizon completion over the same plant states. (a) Disagreement is not uniform over the action space: it peaks on moderate scale-downs and nearly vanishes on large scale-ups. (b) The objective excess is one-sided by construction; no tick favours the greedy completion. (c) Solve time for both, against the control interval.

The disagreement is structured rather than scattered, as Fig. 13(a) shows. It peaks at u*(t) = -2, where the two solvers choose differently on 38.5 % of ticks, and falls to 1.3 % once the exact action is a scale-up of three or more. That shape follows from the mechanism: a large scale-up saturates the rate limit, so every completion agrees on it, whereas a moderate scale-down leaves room for the horizon to be finished several ways and the mispriced tail decides between them. The cases the approximation gets wrong are therefore the ordinary ones, not the extreme ones. Fig. 13(b) shows the excess is one-sided, and Fig. 13(c) the two solve-time distributions against the control interval.

One consequence reaches past the solver and into how the controller is tuned. The control-parameter sweep of Section V-G

was run once under each solver, with the same workloads, the same seeds and the same configuration, so the pair isolates what approximating the solve does to a tuning conclusion. Under the greedy completion, raising the churn weight to unity drove the violation rate to 0.013 % and held it there across every larger value tested. Under Algorithm 2 the same sweep sits near 0.58 %, and the largest divergence over the sweep is 0.013 % against 0.61 % at s = 1.5.

The qualitative reading of that sweep is unchanged: raising s still improves service level, capacity and oscillation together, from 39.5 % over-provisioning and 51.9 oscillations at s = 0 to 22.2 % and 29.3 at s = 1. What was not a property of the controller is the size of the effect. The mechanism is the one Section III-E sets out: greedy completion zeroes the tail, a zero tail misprices precisely the term s scales, and raising s magnifies the mispricing. An approximate solver therefore misleads most where the parameter being tuned is the one the approximation distorts, which is not where a reader would think to look for it.

Exactness costs roughly 72 times the compute of the greedy completion. That multiplier is the real price of the claim. In absolute terms the worst observed solve took 39.4 ms, or 0.79 % of the five-second control interval, so the multiplier is large and the absolute cost is not.

That cost is not always payable. The reduction in Section III-E is strong when the dead time is long relative to the horizon, and the memoised recursion is cheap when the dead time is short enough for the pending queue to collapse distinct prefixes onto one state. Between the two lies a band — a queue long enough to keep states distinct and a horizon long enough to make m large — where neither applies, and there the controller falls back to the greedy completion rather than stall the loop. Every result at the configuration used throughout this paper is solved exactly. The horizon sweep of Section V-A is the one experiment that crosses the band: 46 of its 60 cells are solved exactly and 14 are not, with each run recording which solver produced it. The fallbacks all sit well above the boundary, where the controller is already at its floor; every cell immediately either side of $H\Delta t = \tau_c$ is solved exactly, so the transition itself is measured without the approximation.

J. Robustness to Actuation Delay

Section V-A varied the horizon at fixed cold start. The complementary experiment fixes the horizon at a value that satisfies (6) everywhere and varies the cold start instead, which isolates the controller's tolerance of actuation delay from the question of whether the horizon is adequate. Table XIII reports it and Fig. 14 shows both panels.

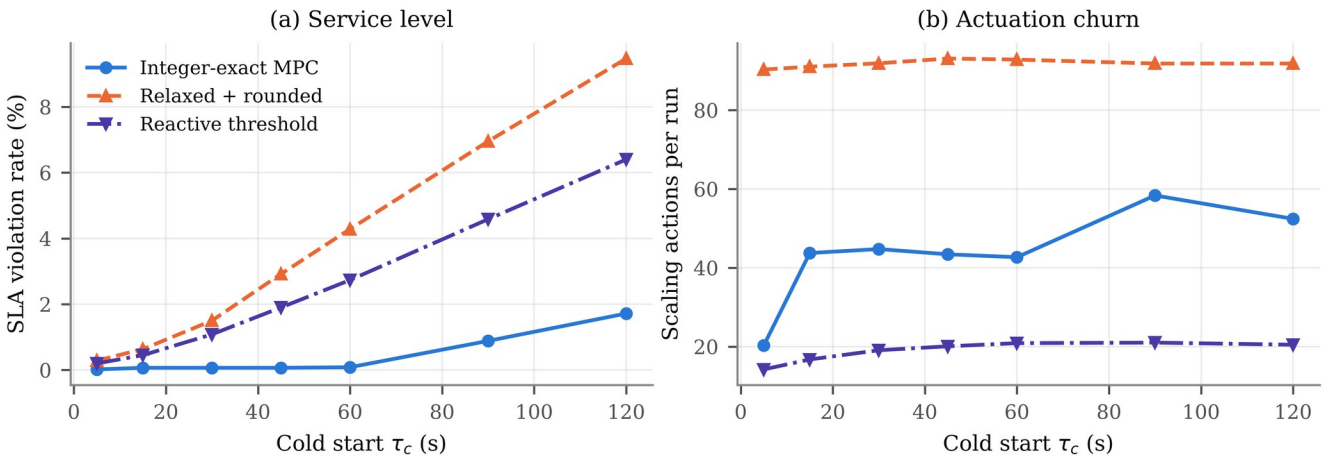
τ_c (s)	Exact MPC viol. (%)	Rounded viol. (%)	Reactive viol. (%)	Exact actions	Rounded actions	Reactive actions
5	0.02	0.28	0.20	20.3	90.3	14.2
15	0.07	0.63	0.45	43.7	91.0	16.7
30	0.07	1.50	1.08	44.7	91.9	19.1
45	0.07	2.92	1.90	43.4	93.1	20.1
60	0.08	4.28	2.73	42.7	92.8	20.9
90	0.88	6.95	4.58	58.3	91.8	21.0
120	1.72	9.47	6.40	52.4	91.8	20.5

Table XIII. Sensitivity to cold-start latency with the horizon fixed at H = 30 (150 s lookahead), which satisfies (6) at every value shown. Synthetic traces, 3 families × 5 seeds = 15 runs per cell.

With an adequate horizon the integer-exact controller is close to insensitive to actuation delay over the practical range. Violations stay between 0.02 % and 0.08 % as the cold start grows from 5 s to 60 s, a factor of twelve in delay. The reactive controller degrades steadily over the same range, from 0.20 % to 2.73 %, and the rounded controller degrades faster still, from 0.28 % to 4.28 %. At the longest delay tested, 120 s, the three sit at 1.72 %, 6.40 % and 9.47 % respectively.

This is the experiment that demonstrates what the ablation of Section V-E could not. In that ablation, disabling the dead-time term changed nothing, because the horizon remained long enough that the optimiser could compensate implicitly. Here the dead time itself is the variable, and the separation between the predictive and reactive controllers grows monotonically with it. Dead-time tolerance is therefore a property of the predictive formulation as a whole rather than of the explicit delay term in (5), and the paper's claim is stated at that level.

The churn panel adds a second observation. The rounded controller's action count is essentially flat at roughly 93 per run regardless of delay, because its oscillation is driven by fractional rounding rather than by the plant. The exact controller's action count varies with delay, as a controller responding to the plant should.



Horizon fixed at $H = 30$ (150 s lookahead), which satisfies Eq. (6) at every τ_c shown. Synthetic traces, 3 families × 5 seeds.

Fig. 14. Service level and churn against cold-start latency at a fixed adequate horizon. Synthetic traces.

K. Per-Workload Behaviour

The aggregate figures of Section V-D average over five demand shapes that stress the controller differently. Table XIV disaggregates them, and the variation is large enough that the aggregate alone would be misleading.

Workload	Method	Violation (%)	Over-prov. (%)	Actions	Oscillations	p95 (ms)
Diurnal	Exact MPC	0.00 ± 0.00	18.4 ± 1.7	37.4	32.1	94
	Rounded	0.30 ± 0.35	3.8 ± 0.3	119.0	109.7	110
	Reactive	0.00 ± 0.00	45.6 ± 13.2	16.4	2.5	89
	Static	0.00 ± 0.00	47.9 ± 0.6	0.0	0.0	84
Burst	Exact MPC	0.00 ± 0.00	22.6 ± 2.4	49.8	35.2	94
	Rounded	5.17 ± 1.18	4.2 ± 0.7	84.3	52.1	168
	Reactive	2.45 ± 0.51	87.1 ± 1.3	27.2	3.0	158
	Static	0.00 ± 0.00	158.4 ± 10.9	1.0	0.0	78
Ramp	Exact MPC	0.00 ± 0.00	13.6 ± 0.8	35.9	25.6	91
	Rounded	0.25 ± 0.29	2.8 ± 0.3	116.0	101.1	107
	Reactive	0.03 ± 0.08	41.3 ± 2.5	15.5	1.0	88
	Static	0.00 ± 0.00	97.2 ± 1.3	1.0	0.0	80
Step	Exact MPC	0.28 ± 0.14	9.9 ± 0.5	32.2	26.9	101
	Rounded	2.38 ± 0.21	3.6 ± 0.3	64.6	57.4	217
	Reactive	2.50 ± 0.00	13.6 ± 1.3	6.7	1.0	213
	Static	0.00 ± 0.00	89.6 ± 8.9	1.0	0.0	81
Sawtooth	Exact MPC	4.00 ± 1.70	52.2 ± 11.5	52.0	30.0	97
	Rounded	0.35 ± 0.58	2.8 ± 0.1	107.5	82.4	105
	Reactive	0.00 ± 0.00	72.6 ± 11.8	18.6	5.4	84
	Static	0.00 ± 0.00	61.8 ± 6.9	1.0	0.0	81

Table XIV. Per-workload breakdown. Synthetic traces, 10 seeds per row, oracle predictor; values are mean ± one standard deviation.

The burst family separates the methods most sharply. There the rounded controller reaches 5.17 % violations against 0.00 % for the exact controller, the largest gap in the table, because a burst drives the pool through many integer boundaries in quick succession and every crossing is an opportunity for the rounding error to change the decision. The diurnal and ramp families, whose demand changes slowly relative to the control interval, show almost no separation at all — 0.30 % against 0.00 % on diurnal traffic. The advantage of exact solution is therefore concentrated in workloads with fast transitions, which is also where an auto-scaler is most needed and least likely to perform well.

The sawtooth family is the one case where the exact controller is worse, at 4.00 % against 0.35 %, and the standard deviation across seeds is correspondingly large at 1.70. A sawtooth resets demand discontinuously every ninety ticks; the churn penalty resists the repeated scale-down that the shape demands, and the controller spends part of each period recovering. This is a genuine failure mode of the churn penalty under periodic discontinuity and is reported as such rather than averaged away.

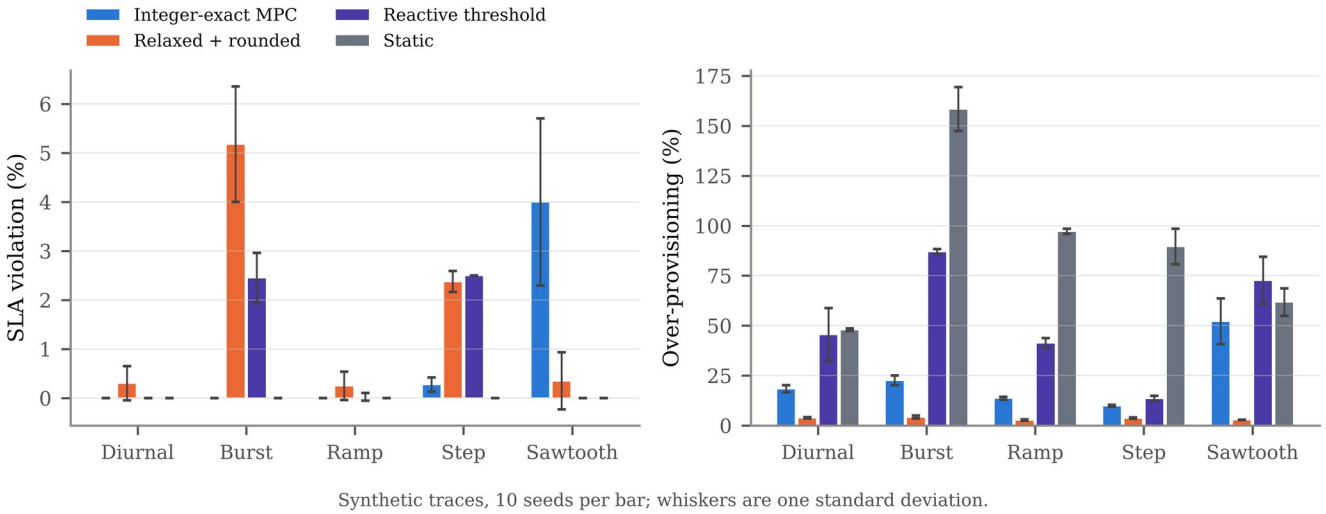


Fig. 15. Per-workload service level and capacity cost. Whiskers are one standard deviation over 10 seeds. Synthetic traces.

L. Decision-Level Analysis

Treating each tick as a three-way classification into scale-up, hold and scale-down against the requirement implied by the observed load gives a finer view than the aggregate rates. Table XV reports it with bootstrap intervals and Fig. 16 shows it. The numbers are low in absolute terms for every method, and the reason is worth stating because it bears on how such metrics should be read.

Method	Scale-up recall	Scale-up precision	Hold F1	Accuracy
Exact MPC (oracle)	0.071 [0.061, 0.083]	0.146 [0.127, 0.166]	0.821	0.695
Exact MPC (persistence)	0.211 [0.198, 0.224]	0.354 [0.324, 0.383]	0.822	0.706
Relaxed + rounded	0.027 [0.018, 0.040]	0.027 [0.018, 0.039]	0.830	0.636
Reactive threshold	0.042 [0.033, 0.052]	0.221 [0.190, 0.255]	0.844	0.732

Table XV. Per-tick decision classification against the observed requirement. Synthetic traces, 50 runs per row; intervals are 95 % bootstrap.

Scale-up recall is low for every method, between 0.027 and 0.211. This is a property of the ground-truth definition rather than of the controllers. The requirement series is computed per tick from the instantaneous arrival rate and therefore changes on most ticks, including by a single replica in response to noise, whereas a well-behaved controller deliberately does not track every such change. A controller that matched this ground truth tick for tick would be the oscillating controller the design is intended to avoid. The metric is reported for completeness and because it is standard in the literature, but it should not be read as a quality ranking, and the paper does not use it as one.

The one comparison it does support is between predictors at fixed control law. Moving from persistence to perfect foresight changes scale-up recall from 0.211 to 0.071 and precision from 0.354 to 0.146. The persistence predictor scores higher on both, which at first appears paradoxical. It follows from the same definitional issue: persistence reacts to the most recent observation and therefore produces decisions that correlate with a per-tick ground truth derived from that same observation, while the oracle acts on demand that has not yet arrived and is scored as wrong until it does. The aggregate service-level figures in Table IV, where the oracle is ahead, are the more meaningful comparison.

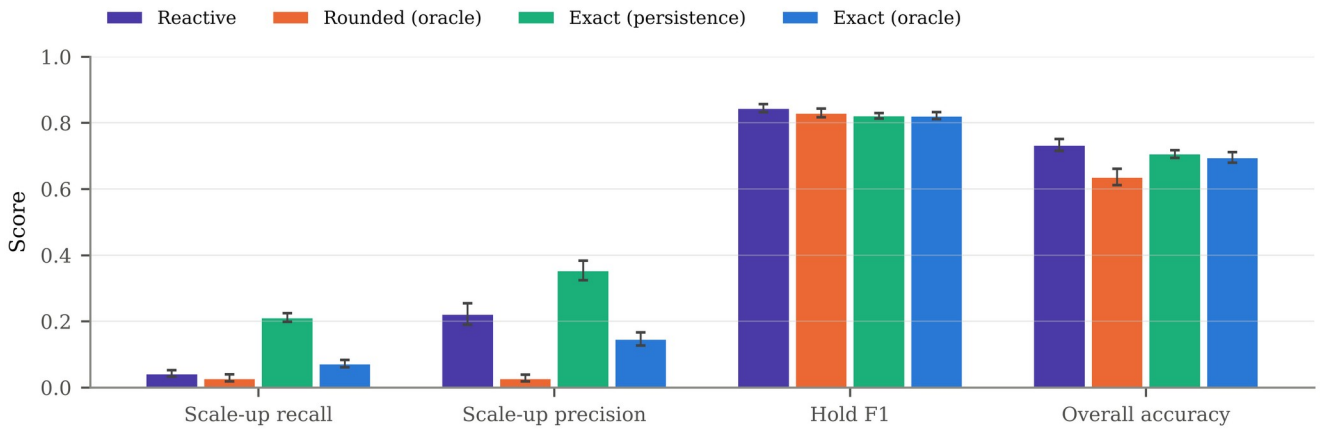


Fig. 16. Per-tick decision classification with 95 % bootstrap intervals. Synthetic traces.

M. Response to Predictor Quality

The bracketing protocol of Section IV-C bounds the predictor's contribution but leaves the interior unmeasured, and a real forecaster lives in that interior. To fill it, a blended predictor was constructed that returns a convex combination of the two brackets, with weight w on perfect foresight and 1 − w on persistence. Sweeping w traces the controller's response to predictor quality without requiring any particular model to be trained. Table XVI reports the sweep and Fig. 17 shows it.

w	Exact MPC violation (%)	Exact MPC over-prov. (%)	Exact MPC actions	Rounded violation (%)	Rounded over-prov. (%)	Rounded actions
0.0	1.07	28.2	55.7	1.76	4.4	98.4
0.2	1.01	25.1	50.9	1.47	4.3	68.8
0.4	0.66	22.3	46.1	1.20	4.1	50.6
0.6	0.33	18.8	39.1	1.21	3.9	50.0
0.8	0.05	17.5	38.3	1.38	3.6	67.1
0.9	0.33	19.1	39.3	1.57	3.4	80.1
1.0	0.97	23.6	43.0	1.83	3.3	94.4

Table XVI. Response to predictor quality. w = 0 is the persistence bracket and w = 1 the perfect-foresight bracket. Synthetic traces, 5 families × 6 seeds = 30 runs per cell.

The response is not monotone, and that is the substantive finding. For the integer-exact controller the violation rate falls from 1.07 % at pure persistence to a minimum of 0.05 % at w = 0.8, then rises again to 0.97 % at perfect foresight. The rounded controller shows the same shape with its minimum at w = 0.5. A perfectly accurate forecast is therefore not the best forecast for either controller on these workloads.

The mechanism is visible in the churn panel of Fig. 17. A perfect forecast reproduces every fluctuation in demand, including the noise, and the controller tracks it; actions rise from 38.3 at the optimum to 43.0 at w = 1. Blending in a persistence component low-pass filters the forecast, and the resulting slight lag behaves like a small damping term. The controller acts less often, holds a steadier pool, and ends up violating less. The effect is stronger for the rounded controller, whose action count more than doubles from 49.2 at its optimum to 94.4 at perfect foresight, because its fractional oscillation compounds with the unfiltered signal.

Two consequences follow for how such systems should be built and evaluated. First, optimising a forecaster for accuracy alone optimises the wrong objective, which is the methodological point the paper makes in Section I and here measures directly: an accuracy improvement that sharpens the predicted signal can degrade the closed loop. Second, the bracketing protocol is conservative in an unexpected direction. Because the interior of the range outperforms both endpoints, the reported bracket does not contain the best achievable result, and a trained forecaster of intermediate sharpness could plausibly perform better than either bracket rather than between them.

The result should not be over-read. It was obtained on synthetic traces whose noise is additive and Gaussian, and the damping benefit depends on that noise being uninformative. On a workload whose high-frequency content carries real demand, filtering it away would cost service level rather than save it. What the experiment establishes is that the relationship between forecast accuracy and control performance is not monotone, not the location of the optimum in general.

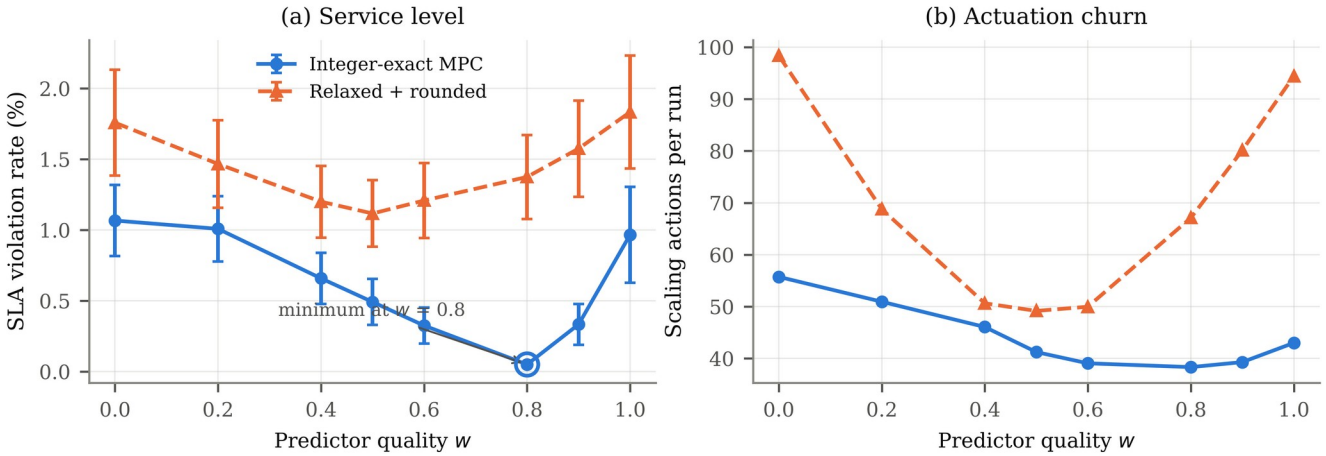


Fig. 17. Controller response to predictor quality. The violation-rate minimum for both controllers lies strictly inside the bracket rather than at perfect foresight. Synthetic traces.

N. Configuration Guidance

The measurements support a short configuration procedure, stated here because the parameters interact and the order matters.

- Measure the cold-start latency of the deployed image, including image pull on a cold cache, readiness probing and load-balancer registration. This single measurement determines the horizon through (6) and is the one parameter that cannot be guessed.
- Set the horizon to the smallest value satisfying $H\Delta t \geq \tau c$, then add one interval of slack. Section V-A shows the penalty for an under-sized horizon is severe and the benefit of oversizing it is negligible, so the asymmetry favours erring long.
- Set the cooldown next. Section V-G shows it is the dominant stability control; 60 s was adequate on every workload tested and longer values continued to help.
- Set the churn weight to approximately unity. Below that value stability degrades; above it nothing changes.
- Leave the deadband at zero or one and the cost asymmetry at any moderate value. Neither was found to matter, and a deadband of two or more trades service level for stability that the cooldown already provides more cheaply.

The procedure assumes a single replica pool with the plant characteristics described in Section IV-A and should be re-validated before transfer to a different deployment, particularly one in which several services share hosts.

O. Decision Trace

Aggregate metrics describe behaviour but not mechanism. Table XVII reproduces sixteen consecutive control ticks from a single run, taken from the rise of the first burst on the burst trace, with every intermediate quantity the controller computes. It is included because the differences the aggregate tables report are visible here as individual decisions, and because an auto-scaler that cannot be read tick by tick cannot be debugged.

Tick	λ (req/s)	Observed u	Forecast u at h=12	Demand Ñ(t+1)	MPC u*	Rounded u	After gates	Applied u	N(t)	Required (ground truth)	Excess
106	121	0.605	0.825	5	0	-1	0	0	6	5	+1
107	113	0.565	0.922	5	0	-1	0	0	6	5	+1
108	124	0.619	0.884	6	0	0	0	0	6	5	+1
109	138	0.690	0.970	6	1	0	1	1	6	6	0
110	133	0.555	0.878	6	0	-1	0	0	7	6	+1
111	152	0.631	0.829	6	0	-1	0	0	7	6	+1
112	145	0.603	0.902	6	0	-1	0	0	7	6	+1
113	141	0.586	0.947	6	1	-1	1	1	7	6	+1
114	145	0.605	0.941	7	-1	-1	0	0	8	6	+2
115	157	0.653	0.911	6	-1	-2	0	0	8	7	+1
116	153	0.638	0.944	7	-1	-1	0	0	8	6	+2
117	157	0.654	0.991	7	-1	-1	0	0	8	7	+1
118	165	0.687	0.948	8	0	0	0	0	8	7	+1
119	184	0.659	0.800	7	0	-1	0	0	8	8	0
120	177	0.631	0.907	8	1	0	1	1	8	7	+1
121	194	0.693	0.843	9	-1	0	0	0	9	8	+1

Table XVII. Sixteen consecutive control ticks from the burst trace, seed 1001, oracle predictor. The ground-truth column is the replica count the instantaneous arrival rate required at the target utilisation; it is computed after the fact and is not available to the controller. The excess column is N(t) minus that requirement.

The two controllers disagree on 11 of the 16 ticks shown, which is consistent with the rates in Table V. The pattern of disagreement is informative: the rounded controller proposes a scale-down on most ticks where the exact controller proposes holding. Its relaxed solution sits slightly below the current count because the instantaneous demand is slightly below it, and rounding converts that fractional preference into a whole replica. The exact controller, evaluating the same horizon under the integrality constraint, finds that removing a replica costs more over the horizon than it saves on the current tick.

The gate column shows the cooldown active on 5 ticks, each time converting a proposed scale-down to a hold. Ticks 113 through 117 are the clearest case: the optimiser proposes removing a replica on five consecutive ticks and the cooldown suppresses all five, because a scale-up occurred recently. The arrival rate then rises through tick 121, which retrospectively makes every one of those suppressed scale-downs the wrong action. This is the mechanism behind the sensitivity result of Section V-G, where the cooldown dominated stability, seen at the level of the individual decisions it changes.

The excess column averages 1.00 replicas over the window, which is the over-provisioning the aggregate tables report expressed per tick. It is positive on almost every tick rather than alternating in sign, which identifies it as a systematic bias of the controller rather than tracking noise. The asymmetric tracking weight of (8) and the cooldown both push in that direction, and Section V-G shows the cooldown is the larger contributor.

One caveat applies to reading this table. The ground-truth column is derived from the instantaneous arrival rate, so it changes on nearly every tick, including by a single replica in response to noise. A controller that matched it exactly would be the oscillating controller the design is intended to avoid, and the excess column should be read as a measure of conservatism rather than of error. This is the same definitional caveat that applies to the decision classification of Section V-L.

P. Positioning Against Prior Work

Table XVIII places the present work against the reviewed systems on the three axes the experiments address. The comparison is of mechanism, not of reported numbers: none of the systems in the table was reimplemented, their published results were obtained on different plants and workloads, and no numerical claim is made against them.

Property	Threshold [1]	Forecast-driven [2,3,8]	Optimisation [4,5,6]	Hybrid [7,11]	This work
Decision variable is integer	yes	varies	no, rounded	varies	yes, solved exactly
Actuation delay modelled	no	no	no	partially	yes, as plant dead time
Horizon tied to delay	n/a	no	no	no	yes, Eq. (6), measured
Decision churn penalised	via stabilisation window	no	no	no	yes, in the objective
Churn reported as a metric	no	no	no	no	yes
Hard pool and rate constraints	yes	varies	yes	varies	yes
Conservatism mechanism	static threshold	none	chance constraint	feedback term	clipped-integral margin
Evaluated on real hardware	yes	varies	yes	yes	no, simulated plant

Table XVIII. Mechanism comparison. Entries describe design choices reported by the cited work, not relative performance. The final row records the principal weakness of the present evaluation.

Three columns separate this work from the rest — exact integrality, a horizon tied to measured delay, and churn treated as both a penalty and a reported metric — and those are precisely the three the experiments quantify. The last row runs the other way: every other system in the comparison was evaluated on real infrastructure, and this one was not.

VI. DISCUSSION

A. What the Measurements Support

Three claims survive the full set of experiments, and it is worth separating them from the claims the design anticipated but the data did not sustain.

The horizon condition is the strongest result and the one with the clearest practical reading. It holds at every cold-start setting tested, the transition lands where (6) predicts rather than at a fixed horizon, and the sign of the predictive controller's advantage reverses across it. It also requires no tuning to exploit: the operator measures one latency and computes one inequality. Section V-J strengthens it by showing that once the horizon is adequate, the controller is nearly insensitive to the delay itself over a factor of twelve.

The integrality result is narrower but holds under both forecaster brackets. Exact and rounded controllers differ on a large and growing fraction of decisions, the difference materialises as churn rather than as violations at moderate operating points, and it materialises as a service collapse at aggressive ones. The statistical picture in Table VIII is consistent with this: the churn difference is unambiguous (Cliff's $\delta = -0.99$, $p < 0.001$) while the violation difference at the default operating point is not significant after correction ($p = 0.122$).

The third claim is about stability mechanisms, and it inverts the design's assumption. The cooldown, which was included as a conventional guard rail, turned out to dominate: removing it raises churn from 42.6 to 267.3 actions per run. The churn penalty in the objective matters too, and improves service level and cost simultaneously rather than trading between them, but it saturates quickly. The deadband and the cost asymmetry, which the design treated as primary controls, were close to inert.

B. What the Measurements Do Not Support

Four anticipated effects were absent, and reporting them is as informative as reporting the successes because each identifies either a redundant mechanism or an untested one.

The explicit dead-time term in the plant model contributed nothing detectable when the horizon was already adequate. The optimiser compensates for delay implicitly through the horizon, and the delay term is a formal convenience rather than a separate source of benefit. The paper therefore attributes dead-time tolerance to the predictive formulation as a whole.

The adaptive margin never moved. Under a perfect predictor the tracking error is negligible, and under a memoryless one it is large but zero-mean, so the clipped integral has nothing to integrate in either case. The mechanism is untested rather than unnecessary, and testing it requires a predictor whose error has sustained sign, which is exactly what a drifting learned model would produce and neither bracket does.

The safety ceiling never activated at any corruption level tested. The same caveat applies: symmetric multiplicative noise does not produce the sustained under-prediction the ceiling exists to catch. Section V-F shows noise actually improved service level by inflating capacity through the asymmetric cost, which is a property of the objective rather than evidence of robustness.

Finally, the proposed controller did not dominate. It carries more capacity than the rounded controller at matched settings (23.3 % against 3.5 %) and issues more scaling actions than the reactive baseline (41.5 against 16.9). An operator whose workload resembles the slow families in Table XIV, and who can tolerate the rounded controller's violation rate, has no reason to prefer exact solution on these results.

C. When Each Mechanism Is Worth Its Cost

Synthesising across the experiments gives a decision rule that is narrower than a general recommendation but better supported than one. Exact integer solution is worth adopting when the pool is small relative to demand swings, when the target utilisation is aggressive, or when the workload has fast transitions — the three conditions under which Table V, Table VI and Table XIV respectively show the largest separation. It is not worth adopting on slowly varying traffic at conservative targets, where the measurements show no advantage and a capacity penalty.

Predictive scaling in general is worth adopting when the actuation delay is long relative to the control interval, which Section V-J quantifies, and only when the horizon can be configured to cover it. Where the delay is short — at $\tau c = 5$ s the reactive baseline violates on 0.17 % of ticks against 0.05 % predictive — the additional machinery buys little.

VII. LIMITATIONS AND THREATS TO VALIDITY

A. Construct Validity

Construct validity asks whether the quantities measured correspond to the properties claimed. Two constructs in this paper are imperfect and the imperfection bounds the claims that rest on them.

The service-level violation rate counts ticks whose modelled tail latency exceeds the objective. Real service-level objectives are evaluated over windows and against measured request latencies, not over control ticks against a queueing approximation. A controller that performs well on this construct would perform well on the real one only to the extent the approximation holds, which Section IV-C argues it does in the moderate-utilisation regime and does not near saturation, where the measure is censored.

The ground-truth replica requirement, used by both the decision classification of Section V-L and the excess column of Table XVII, is computed from the instantaneous arrival rate. It therefore treats noise as demand and changes on nearly every tick. It is an adequate construct for measuring systematic bias, which is how this paper uses it, and an inadequate one for ranking controllers by decision quality, which is why the classification results are reported but not used to support any claim.

B. Internal Validity

Internal validity asks whether the measured differences are caused by the manipulated variable. The experimental design supports this reasonably well. Every method sees the identical trace for a given family and seed, so comparisons are paired and the Wilcoxon test is appropriate. Each ablation and sweep changes exactly one component or parameter with the others held fixed. The plant is shared across methods within a run, so differences cannot arise from plant variation.

One threat remains. The methods operate at different default utilisations, so the aggregate comparison of Table VII compares points rather than frontiers. This is the reason the operating-point sweep of Section V-C exists, and conclusions about service level are drawn from that sweep rather than from the aggregate table.

C. External Validity

External validity asks how far the results transfer, and it is where this evaluation is weakest. The plant is simulated, the workloads are synthetic, the learned forecaster was never run, and a single replica pool was controlled. Each is enumerated in the following subsection with the specific claim it bounds.

One form of transfer is better supported than the others. The horizon condition of (6) is a statement about the relationship between two latencies, and the experiment confirms the transition occurs where that relationship predicts rather than at a fixed horizon. A result that tracks a parameter in this way is more likely to transfer than one that holds only at the values tested, because the mechanism rather than the magnitude is what has been identified. The integrality result is in the opposite position: the magnitude of the rounding gap depends on pool size, demand volatility and the operating point, and Table V shows it varying by more than a factor of three across pool sizes alone.

D. Enumerated Limitations

This evaluation has substantial limitations. They are listed here rather than distributed through the text because several of them bound the claims directly.

Simulated plant only. No result in this paper was obtained on real containers. The plant is the analytical queueing simulator described in Section IV-A, and the cold-start interval is a parameter of that simulator rather than a measurement. Image pull variance, scheduler interference, connection draining and load-balancer registration delay are all absent. The horizon–dead-time law of Section V-A is a property of the model as much as of the controller, and confirming it on a real Docker deployment is the single most important piece of outstanding work.

Synthetic workloads. The five trace families are generated, not recorded. Public traces could not be retrieved because the environment has no network access. Real traffic has heavier tails and non-stationarity that these traces do not reproduce.

The learned forecaster was not executed. PyTorch is unavailable in the environment and could not be installed, so the temporal convolutional forecaster, and the recurrent and attention-based baselines that would be compared against it, were not run. The bracketing protocol of Section IV-C bounds the effect of this substitution but does not eliminate it: no claim about forecasting accuracy is made anywhere in this paper, and the comparison against statistical and deep forecasting baselines remains outstanding. For the same reason the ARIMA baseline, which requires statsmodels, was not run.

Two mechanisms are unexercised. The adaptive margin never moved from its initial value and the safety ceiling never activated. Both are implemented and both are inert under the conditions tested. They should be regarded as untested rather than validated.

Censored latency. The heavy-traffic approximation diverges when offered load exceeds pool capacity. Such ticks are censored at a finite ceiling, which compresses the upper tail of the latency distribution and makes the reported mean latency a lower bound in saturated conditions.

Single service. Only one replica pool is controlled. Real deployments comprise interdependent services sharing hosts,

and the coupling between their scaling decisions is not modelled.

Baseline fidelity. The relaxed-and-rounded controller reproduces the integrality choice made by published optimisation-based auto-scalers on an otherwise identical objective; it is not a reimplementation of any specific published system, and it omits the chance constraints that some of them use. It should be read as an ablation of the integrality constraint rather than as a competing system.

VIII. CONCLUSION

Three properties of predictive container auto-scaling were measured on a simulated pool, with the predictor held at two extremes so that the control law was the only variable. They differ in how firmly they are established, and the distinction is worth keeping.

The forecast horizon must cover the actuation dead time. Across the 5 cold-start settings sampling both sides of the boundary, the violation rate fell from 2.92 % to 0.27 % as the horizon crossed $H\Delta t = \tau c$, and below that boundary the predictive controller performed worse than a reactive baseline by up to 3.69 percentage points. Horizon selection is therefore not a tuning preference but a feasibility condition, and it can be checked before deployment from a single measurement of cold-start latency. This is the firmest of the three results: the transition appeared at every cold-start setting tested and landed where the inequality predicts rather than at a fixed horizon.

The integer problem does not have to be approximated. Dead time confines the tracking-relevant decisions to the first $H - \tau + 1$ steps of the horizon and leaves the rest separable, which reduced the search to 4 nested integer choices at the configuration used here. The solver built on that reduction reproduced the brute-force optimum in all 400 states small enough to enumerate exhaustively, which is a verification rather than a proof but a stronger warrant than the argument it replaces. Completing the horizon greedily instead, a device sometimes called exact on the grounds that only the first action is applied, returned an optimal first action in only 354 of those 400 states, and at the paper configuration differed from the optimum on 10.83 % of the plant states drawn, always in the direction of higher cost.

Relaxing the integrality of the scaling decision changes behaviour measurably. The relaxed controller disagreed with the exact one on 19.4 to 71.4 % of control ticks, issued up to 3.4 times as many scaling actions, and degraded from 0.45 % to 17.92 % violations as the utilisation target was raised, against 0.00 % to 3.58 % for the exact controller. Exact solution cost at most 43.18 ms per tick, 0.86 % of the control interval, so relaxation buys compute that is not scarce at these problem sizes. That is a claim about this configuration rather than a general one: the exact solve is cheap here because the dead time is long relative to the horizon, and a pool with near-instant start-up and a long horizon would not enjoy the same reduction.

The results also qualify the approach. At the default operating point the exact controller did not significantly reduce violations relative to either the rounded or the reactive baseline, and it carried more capacity than the rounded controller to achieve the service level it did. Its measured advantages are in actuation stability and in graceful degradation at aggressive operating points. Two of its components, the adaptive margin and the safety ceiling, were inert throughout.

The immediate priorities for further work follow from Section VII: reproducing the horizon–dead-time transition on a real Docker deployment with a measured cold-start interval, repeating the comparison with a trained forecaster in place of the brackets, and designing a corruption model with sustained bias that would actually exercise the margin adaptation and the safety ceiling.

REFERENCES

[1] T. Van Do et al., "Properties of horizontal pod autoscaling algorithms and application for scaling cloud-native network functions," preprint, 2025.

[2] B. Jeong and Y.-S. Jeong, "ARAScaler: Adaptive resource autoscaling scheme using ETimeMixer for efficient cloud-native computing," IEEE Trans. Services Computing, vol. 18, no. 1, pp. 72–84, 2025, doi: 10.1109/TSC.2024.3522815.

[3] J. W. Fog, J. J. T. Møller, T. M. Jensen, D. Taibi, and M. Albano, "Comparing neural and statistical time-series models for proactive auto-scaling in Kubernetes," in Proc. IEEE Int. Conf. Service-Oriented System Engineering (SOSE), 2025, pp. 151–161, doi: 10.1109/SOSE67019.2025.00022.

[4] E. Incerto, M. Tribastone, and C. Trubiani, "Combined vertical and horizontal autoscaling through model predictive control," in Euro-Par 2018: Parallel Processing, Lecture Notes in Computer Science, vol. 11014, Springer, 2018, pp. 147–159, doi: 10.1007/978-3-319-96983-1_11.

[5] G. Quattrocchi, E. Incerto, R. Pinciroli, C. Trubiani, and L. Baresi, "Autoscaling solutions for cloud applications under dynamic workloads," IEEE Trans. Services Computing, vol. 17, no. 3, pp. 804–820, 2024, doi: 10.1109/TSC.2024.3354062.

[6] D. Zou, W. Lu, Z. Zhu, X. Lu, J. Zhou, X. Wang, K. Liu, K. Wang, R. Sun, and H. Wang, "OptScaler: A collaborative framework for robust autoscaling in the cloud," Proc. VLDB Endowment, vol. 17, no. 12, pp. 4090–4103, 2024, doi: 10.14778/3685800.3685829.

[7] Z. Cai et al., "Deep learning and feedback control based container auto-scaling for cloud native microservices," IEEE Trans. Services Computing, 2025.

[8] F. Meng et al., "CATScaler: A convolution-augmented transformer scaling framework for cloud-native applications," IEEE Trans. Services Computing, 2025.

[9] Z. Fang, H. Ma, G. Chen, and R. Buyya, "HGraphScale: Hierarchical graph learning for autoscaling microservice applications in container-based cloud computing," IEEE Trans. Services Computing, 2025.

[10] B. Kumar, A. Verma, and R. Buyya, "Predictive autoscaling in cloud-native and federated cloud-edge computing environments: A taxonomy and future directions," arXiv:2606.07046, 2026.